# PHP + WordPress 01

PHP + Wordpress/ CMSを活用したWEBサイト制作



Re start

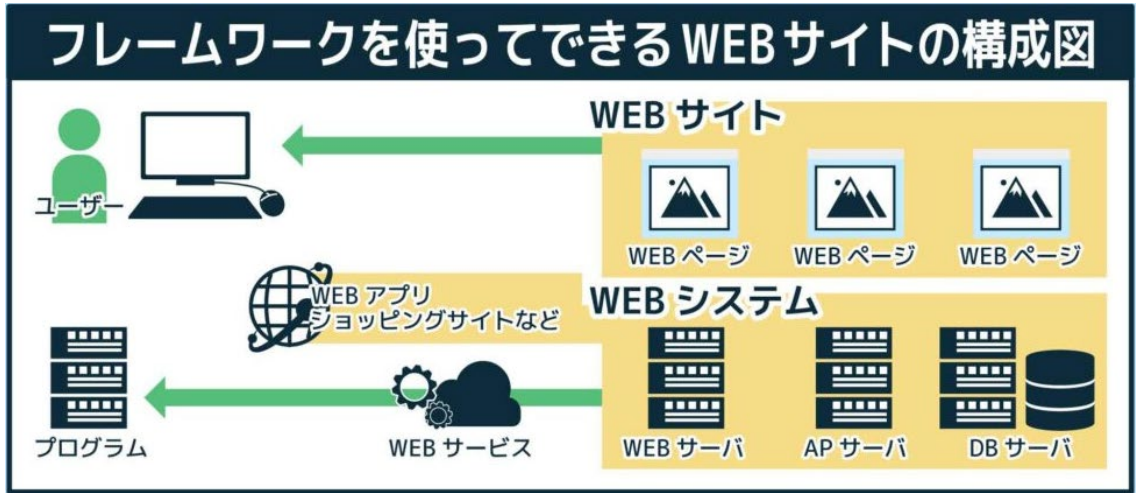
## PHPとは

PHPとは、Webアプリケーションや高度なWeb制作に使用されるプログラミング言語です。
PHPは初心者でも習得しやすいのが特徴で、多くの開発現場で使われています。

PHPの歴史は古く、第一線の言語として活躍しています。
そのため、インターネット上に開発に関する情報が多く、サンプルとなるコードや関数、
ライブラリも豊富にあります。

また、世界中で最も使われているCMSツールである**WordPress**にもPHPが使われています。

問い合わせフォーム、ログイン機能、
予約機能、ブログ機能など、高度な
機能をもつWebサイトを作りたい場合、
WordPressをPHPでカスタムしながら
作るのが効率的です。



## PHPのデータ型

**PHPにもJavaScriptと同じように、データ型が存在します。**

| データ型 | データ型(英語) | 説明 |
|---|---|---|
| **整数** | int,integer | -1 , 0 , 1などの整数 |
| **小数** | flaot,double | 3.14159のような小数 |
| **文字列** | string | “”でかこまれた文字 |
| **論理** | bool,boolean | true, false |

**上の表のとおり、JavaScriptと同じです。**
**このデータ型は他のプログラミング言語にもデータ型は存在し、ほぼ同じ内容です。**



Re start

## PHPの変数

JavaScriptには変数の宣言がありましたが、PHPでは変数を宣言する必要がありません。

JavaScript

```
let 変数名;
変数名 = "値";
```

PHP

```
$変数名 = "値";
```

$のあとに変数名を書いて、値を代入すれば変数として機能します。
変数の特徴や機能はJavaScriptと同様で、まったく同じ感覚で使えます。



Re start

## PHPの命名規則

Javascriptで変数名や関数名をつける際に、単語の区切りを大文字にするキャメルケースが一般的です。

PHPでは、区切りを大文字にするキャメルケースと区切りを _(アンダーバー)にするスネークケースのどちらかが使われます。
開発現場のコーディング規約に沿って、正しい方を使いましょう。

WordPressでは変数名も関数名もスネークケースが使われているので、スネークケースで記述するのが一般的です。

キャメルケース

```
$variableName = "変数名";
```

PHP

```
$variable_name = "変数名";
```



Re start

## PHPの文字列

PHPでも文字列を結合することができます。

JavaScript

```
let examStr = "Hello" + " World!";
```

PHP

```
$examStr = "Hello"."World!";
```

PHPで文字列を結合する場合は、+ ではなく . を使って結合します。
その他、文字列の分割などの操作を行うための関数も多数用意されています。



Re start

## PHPの配列

**PHPでの配列の操作は、以下のように記述します。**

```
// 配列の宣言・初期化
$sports_arr = [
    '野球',
    'バドミントン',
    'バスケットボール'
    ];

// 配列の要素を取得
$sports_arr[0]; // 野球

// 配列の要素を追加
$sports_arr[] = 'サッカー'; // 末尾に追加

// 配列の要素を削除
unset($sports_arr[1]); // 指定したインデックスの要素を削除

// 配列の要素を変更
$sports_arr[0] = 'サッカー'; // 指定したインデックスの要素を変更
```



Re start

## PHPの連想配列

通常の配列は値を格納するとインデックスが割り振られますが、連想配列はキーとキーに対応した値を格納することで、キーを指定して値を参照することができます。

```
// 連想配列の宣言・初期化
$sports_arr = [
    'Baseball'=>'野球',
    'Badminton'=>'バドミントン',
    'Basketball'=>'バスケットボール'
    ];

// 連想配列の要素を取得
$sports_arr['Baseball']; // 野球

// 連想配列の要素を追加
$sports_arr['Soccer'] = 'サッカー';

// 連想配列の要素を更新
$sports_arr['Baseball'] = 'ベースボール';
```

このようにキーを自由に設定できるので、何の値なのかわかりやすいですね。



Re start

## PHPの条件分岐・ループ

PHPでも if のような条件分岐や、forのようなループはそのまま存在します。

```
// 条件分岐
if ($lucky_number == 7) {
    $result = "Lucky number is 7";
} else {
    $result = "Lucky number is not 7";
}
// ループ処理
for ($i = 0; $i < 10; $i++) {
    $result = $i;
}
```

このように、変数の記述こそ違いますが、それ以外はJavaScriptと一緒です。



Re start

## PHPの関数

関数についても、JavaScriptと同じ方法で定義することができます。

```
// 関数の定義
function get_Lucky_number() {
    return 7;
}

// 関数の呼び出し
$lucky_number = get_Lucky_number();

//引数のある関数の定義
function get_number2($number) {
    $number = $number + 1;
    return $number;
}
```

PHPとJavaScriptは非常に似ている言語であることが分かりました。

実はPHPとJavaScriptだけでなく、他のプログラミング言語も似ている部分が多くあります。
1つのプログラミング言語を習得すると、他の言語を習得するのに時間がかからないと
言われているのはその為です。



Re start

## WordPressとは

WordPressとは、ブログやウェブサイトの作成や、記事、コンテンツの管理ができる
Webコンテンツ管理システム(CMS)です。
無料で自由に利用ができ、再配布も可能です。
テーマと呼ばれるテンプレート（雛形）があれば、プログラミングの知識なく
ブログやWebサイトを作成することができます。

WordPressは、世界で最も使われているCMSで、シェアは60%以上といわれています。

人気がある反面、クラッカー(悪意をもってデータを破壊する者)に狙われやすく、
WordPressでWebサイトを構築する場合は、セキュリティ対策が必須となります。

WORDPRESS



Re start

## CMSとは

CMSとはコンテンツ・マネジメント・システムの略で、Webサイトのコンテンツを管理するためのシステムです。
専門的な知識がなくても簡単にWebサイトを作成、編集できるのが特徴です。

世界で最も使われているCMSはWordPressですが、他にもたくさんのCMSがあります。
無料のCMSから有料のCMSまで多数の種類がありますが、それぞれ取り扱いの方法が全く違うというデメリットもあります。

WordPressは世界中で使われているので、インターネット上には多くの情報が公開されています。
プログラミング言語と同じように、情報が多い事はCMSを選ぶうえで大きなメリットになります。

## Localとは

WordPressは、サーバーにアップしないと動かないシステムで、開発には不便です。
LocalというツールはPC上にサーバーやデータベースの機能を持たせ、サーバーにアップせずに
WordPressを動かすことができるツールです。

このLocalを使えば、変更毎にサーバーにアップせずに済むので効率が上がります。
また、本来かかるサーバーの費用もかからないので、費用の節約にもなります。

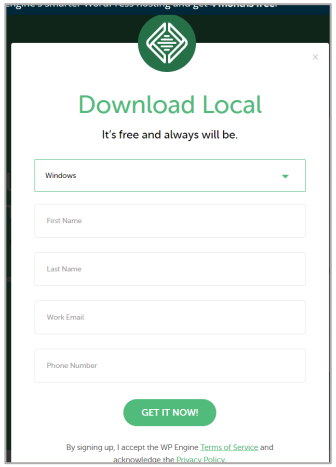
LOCAL
by Flywheel



Re start

# Local環境をつくる

## ① Localをダウンロード




https://localwp.com/
↑のURLにアクセス。
OSを選択、メールアドレスを入力してダウンロード。

## ② Localをインストールする

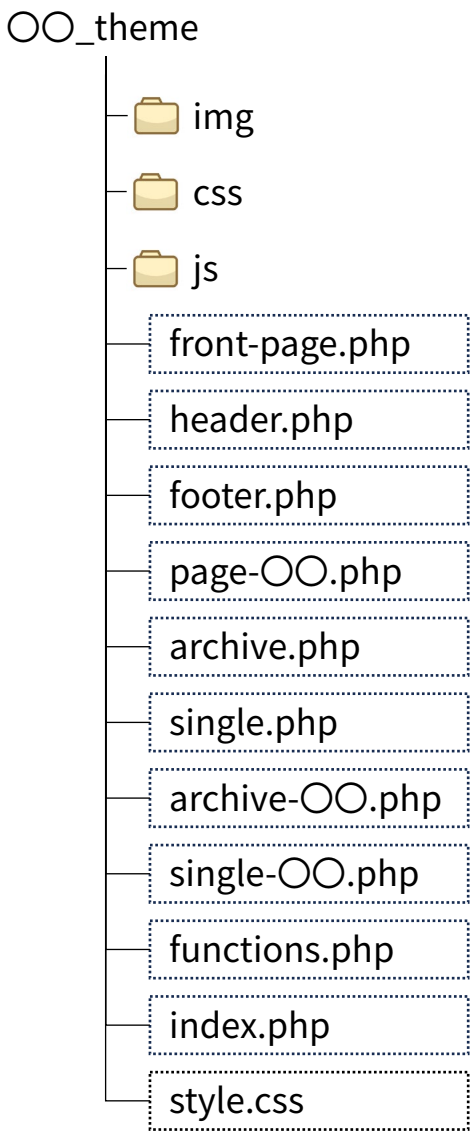
次へを押してインストール。
完了をクリックして終了。

## テーマとは

WordPressには、テーマという仕組みがあります。
テーマとは、HTMLやCSSなどでデザインしたサイトを編集するために必要なファイルをひとまとめにしたものです。
いくつかのテンプレート（編集・調整が可能な雛形のようなもの）で構成されていて、画像や文章など、編集が可能な部分であれば自由に編集ができます。

WordPressでは、テーマを設定することでWebサイトを表示することができます。
HTML/CSSなどのソースコードを触ることなく、専門知識なしで編集することができます。

有料から無料のものまで多数のテーマが配布されており、簡単に公開ができますが、
専門知識がないと決められた部分のみしか編集できないというデメリットもあります。



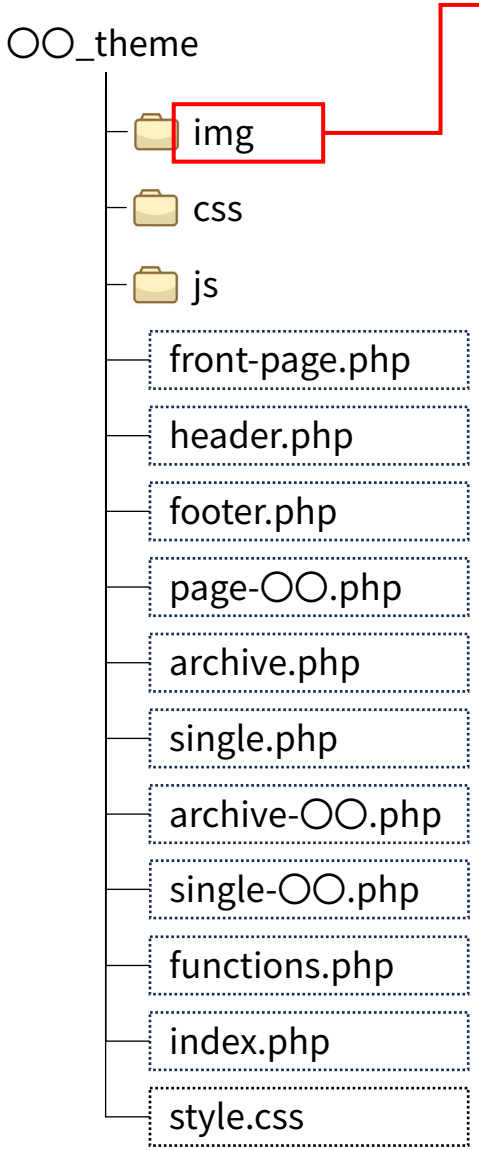

## テーマのファイル構造

WordPressのテーマの構造にはルールがあり、その通りに作らないと正常に表示されません。

HTML/CSSで見慣れた .html というファイルが見当たらず、かわりに〇〇.phpというファイルがたくさん並んでいますね。

WordPressはページの種類ごとにテンプレート（雛形）を使い分ける仕組みになっていて、このphpファイルがそれぞれテンプレートをもっています。

このように、ページ毎に用意されるテンプレートのことを、テンプレートファイル といいます。



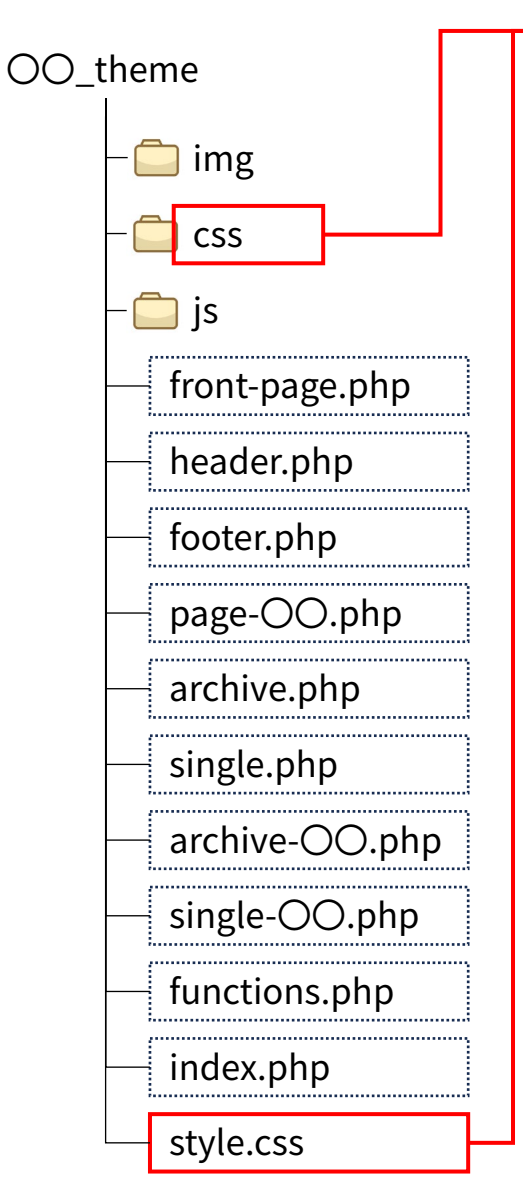

## imgフォルダ

imgフォルダには、画像や動画などの素材を格納します。

HTML/CSSでWebサイトを作成する際もフォルダで切り分けますが、
それと同じようにフォルダで素材をひとまとめにします。

ある程度の規模以上のサイトを作成する場合、素材の数が多くなり
目当ての素材を見つけるのに時間がかかることがあります。

素材の数が多い場合は、このimgの中にさらにフォルダを作り、
素材の種類ごとにまとめると作成の効率が上がります。

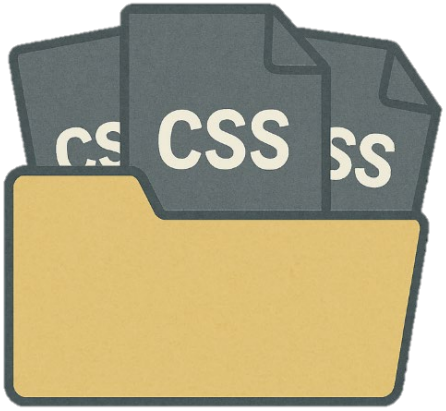


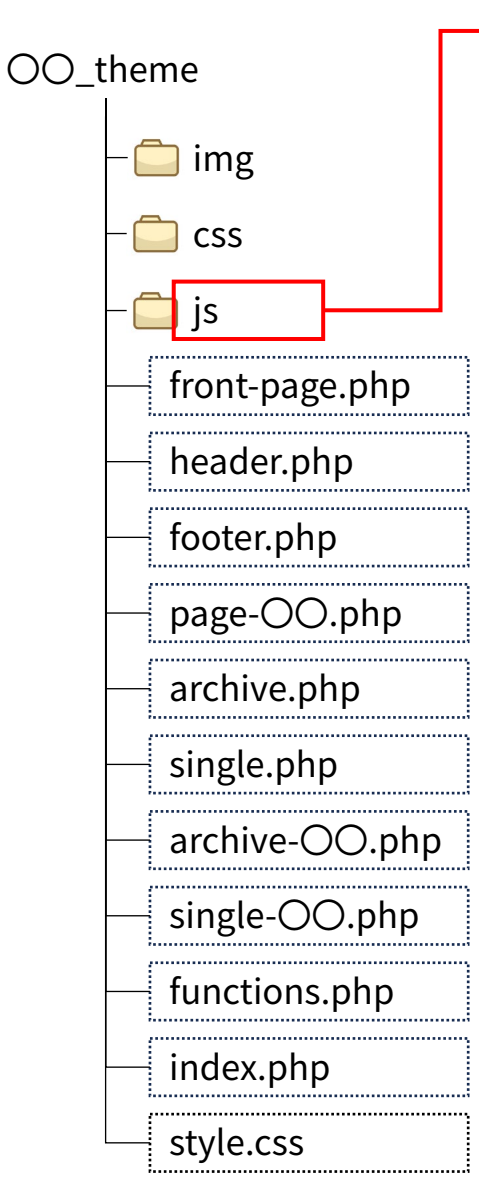

## cssフォルダ

cssフォルダには .css ファイルを格納します。

WordPressでは、index.php と style.cssがテーマフォルダの直下にないと、テーマとして認識されません。
そのため、style.cssはthemeフォルダの直下に配置してください。
各テンプレート共通のCSSはstyle.css に記述しましょう。

それ以外の各ページごとのCSSは、cssフォルダに格納します。
.cssのファイル名の命名規則については後述します。

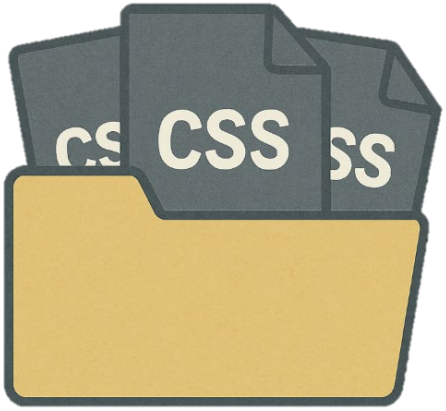


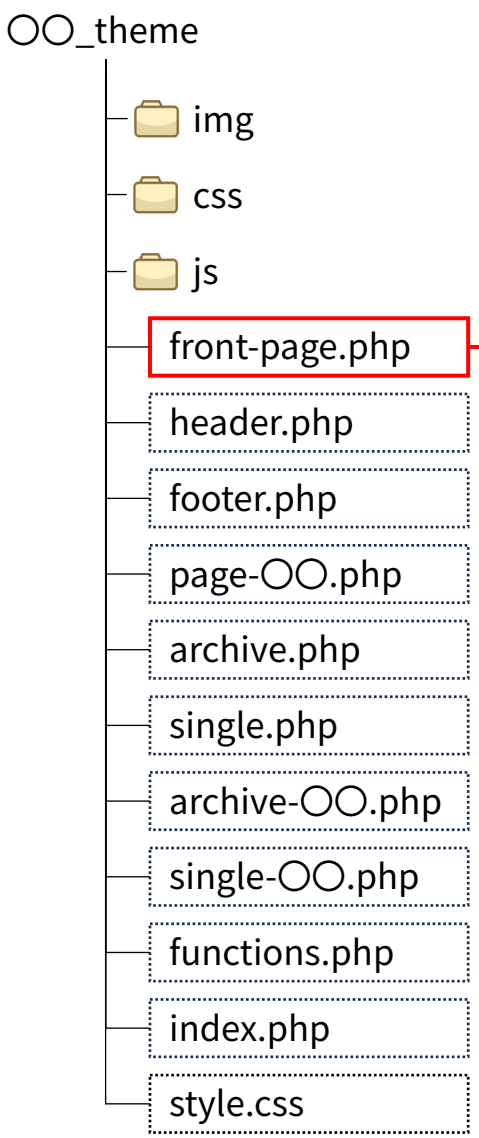



## jsフォルダ

jsフォルダには .js ファイルを格納します。

WordPressでもJavaScriptは通常通り動きます。
ライブラリなどのファイルもこのフォルダに格納します。



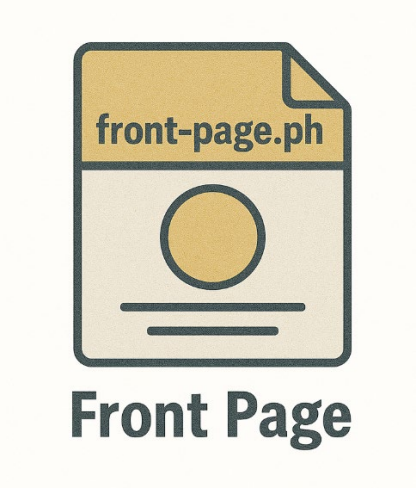


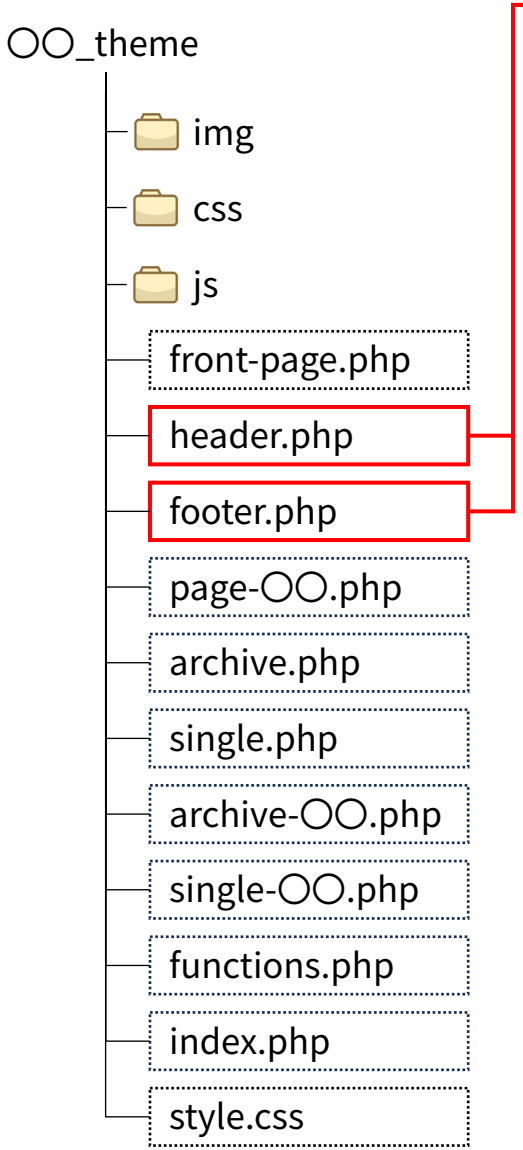

## front-page.php

HTML/CSSでWebサイトを作る際、トップページの内容はindex.htmlに記述していましたが、この構造の場合はfront-page.php にトップページの内容を記述します。

index.php も存在はしていますが、テーマの認識のために配置していて、何も記述しません。

WordPressのトップページのテンプレートファイルを作成する場合は、front-page.phpと命名しましょう。



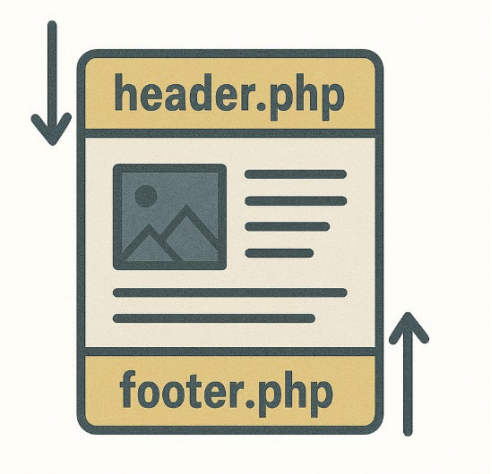


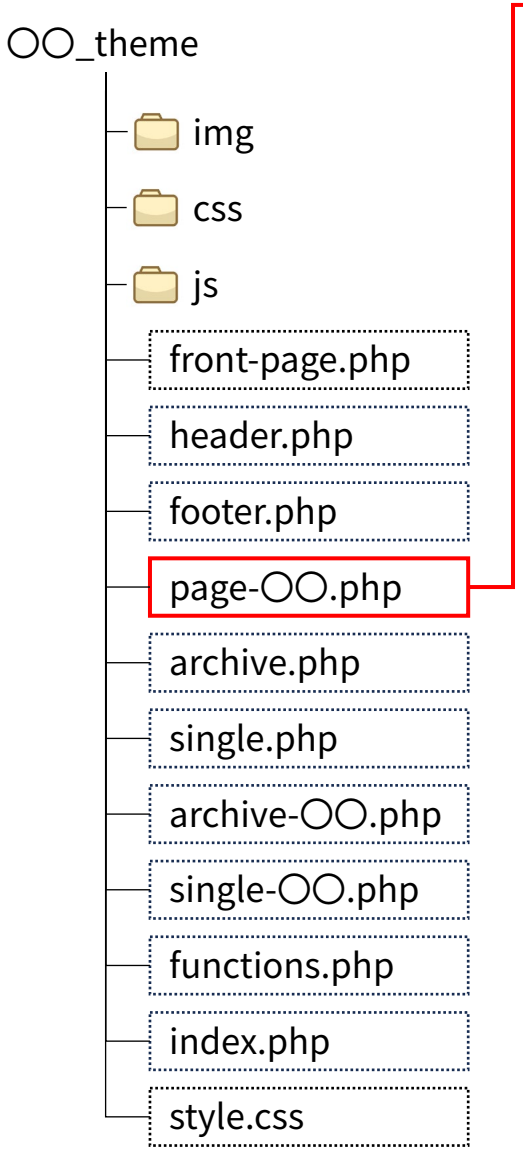

## header.phpとfooter.php

header.php と footer.php は、その名の通りそれぞれヘッダーとフッターを記述するテンプレートファイルです。

HTML/CSSでヘッダーとフッターを記述するときは、それぞれのページに記述していました。
WordPressでは、ヘッダーとフッターはパーツとして分かれていて、各ページに呼び出されて表示されます。

ということは、他のページにはヘッダーとフッターは記述しません。
呼び出しの処理のみ記述すればいいので、効率的ですね。






## 固定ページ

トップページ以外の独立したコンテンツを持っている下層ページの
のことを、固定ページと呼びます。
例えば会社概要やプライバシーポリシー、問い合わせページなどです。

固定ページを作るときは、page-〇〇.php とファイル名をつけます。
この〇〇の部分には、ページ内容を表す英単語を入れます。

例えば問い合わせページであれば page-contact.php、
プライバシーポリシーであれば page-privacypolicy.php などです。

基本的にはこのファイル名は、デザインの時点で指定されるので、
デザインにしたがった名前でファイルを作成しましょう。

- ○○_theme
  - img
  - css
  - js
  - front-page.php
  - header.php
  - footer.php
  - page-○○.php
  - archive.php
  - single.php
  - archive-○○.php
  - single-○○.php
  - functions.php
  - index.php
  - style.css

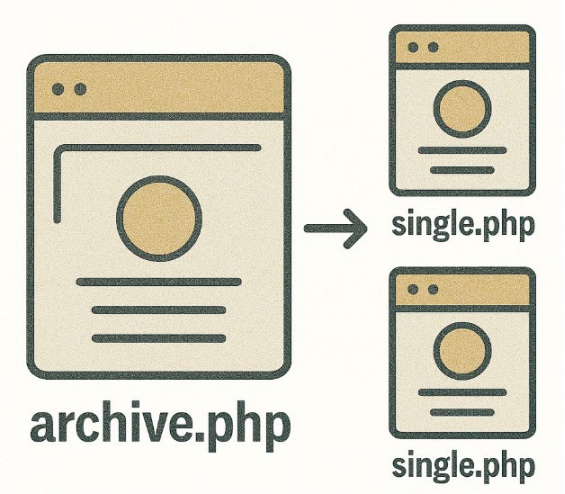
## archive.php

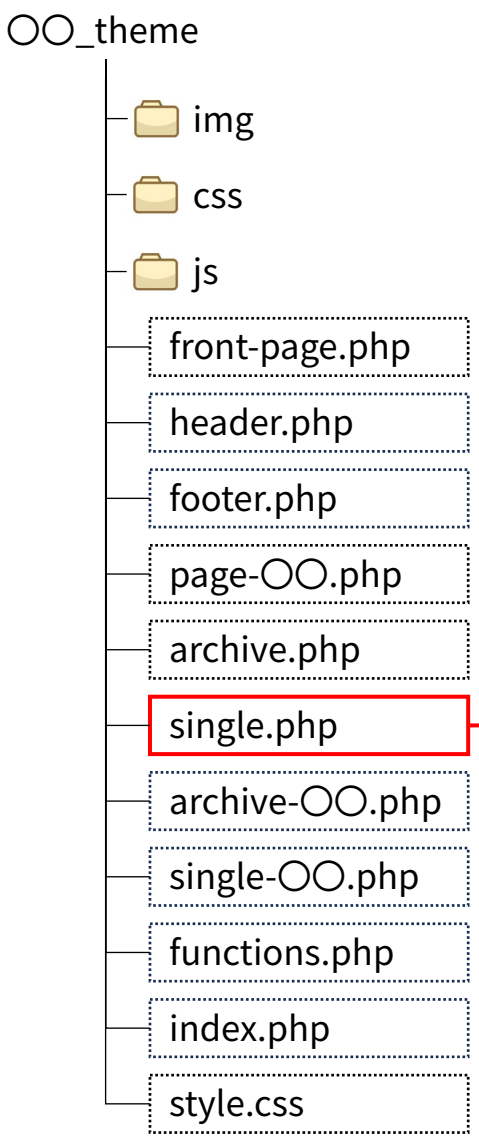
WordPressには、ブログのように記事を投稿できる機能を簡単に追加できます。

この記事の一覧を表示するのが、archive.phpです。

カテゴリーやタグでの絞り込み、表示する内容など、自由に設定できます。

このarchive.phpは、主にお知らせやニュースなどで使用されます。

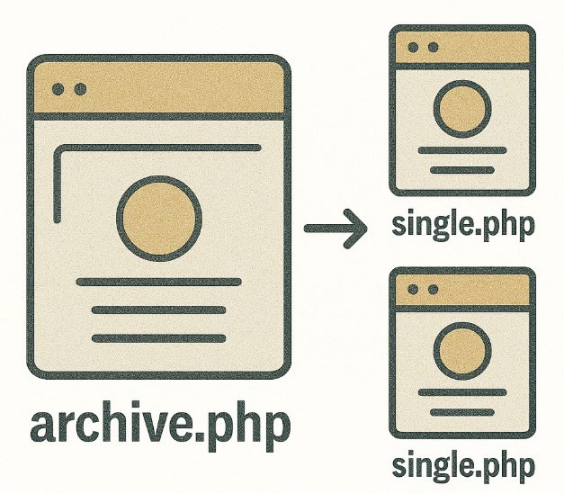


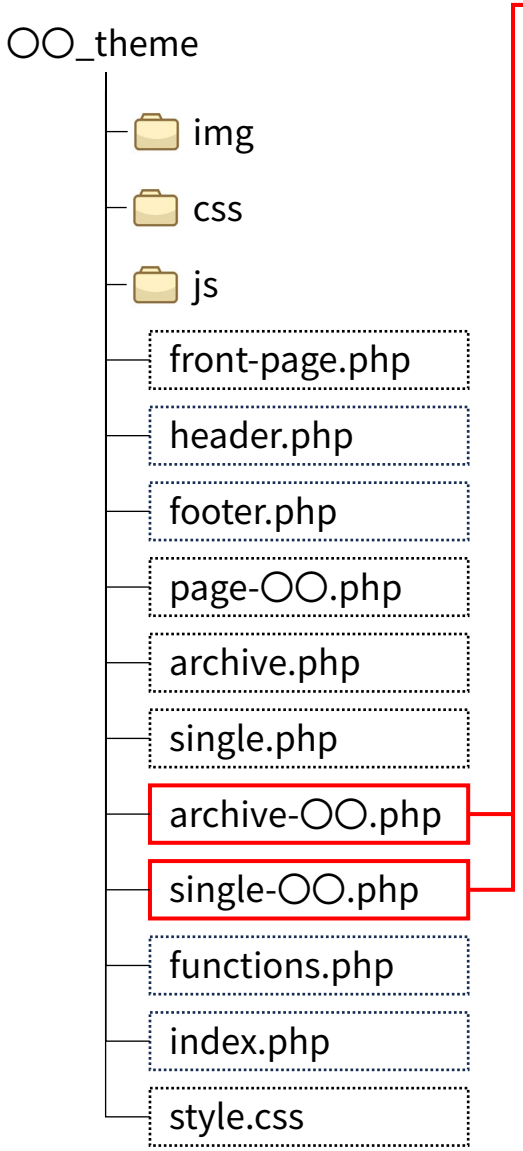



## single.php

archive.php で記事の一覧を表示できますが、その記事の詳細はこの single.php で表示されます。

このsingle.php は記事の雛形のようなもので、ここにデザインを設定すれば、投稿した記事全てにデザインが反映されます。



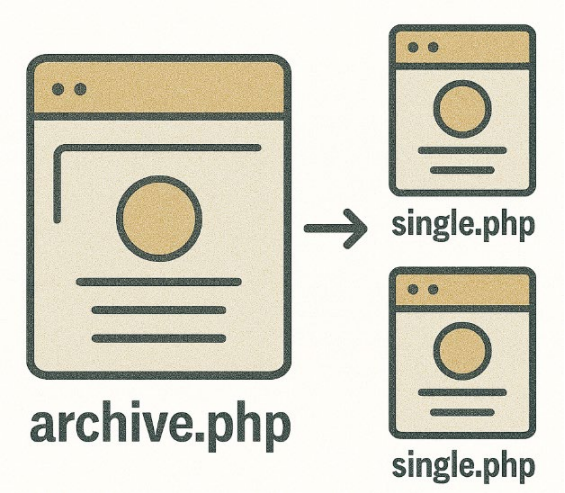


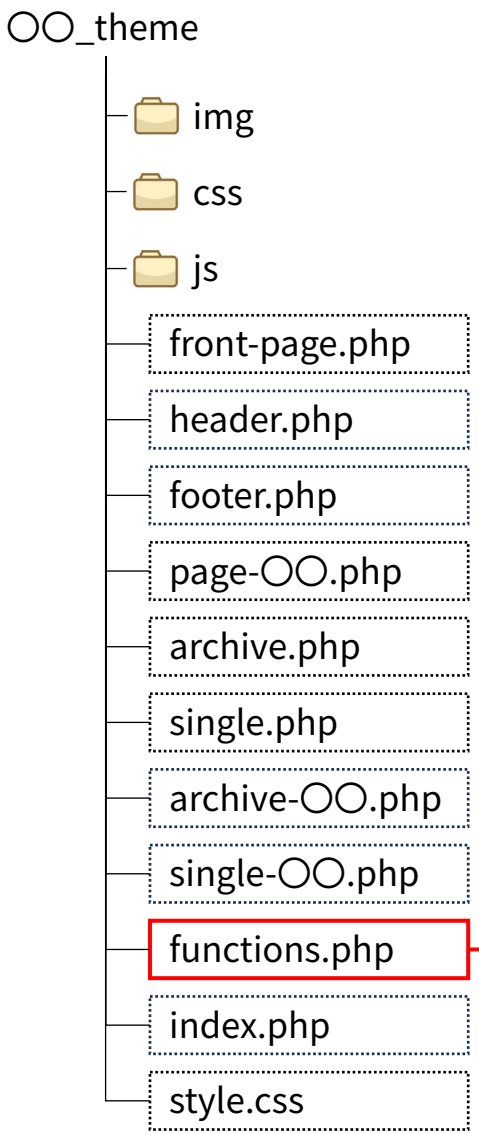

## archive-〇〇.phpとsingle-〇〇.php

Webサイトには、お知らせやニュース以外にも投稿機能が必要な場合があります。

例えば商品の一覧や店舗の一覧など、archive.phpやsingle.phpとは違うデザインで表示したい場合、このように別のテンプレートとしてファイルを作成します。

この〇〇の部分には、固定ページと同じように英単語をいれます。
これもデザインで指定される事が多いので、デザインに従いましょう。



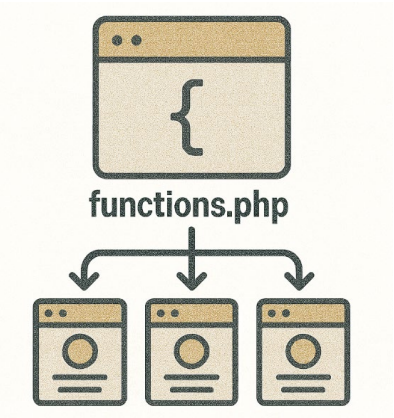


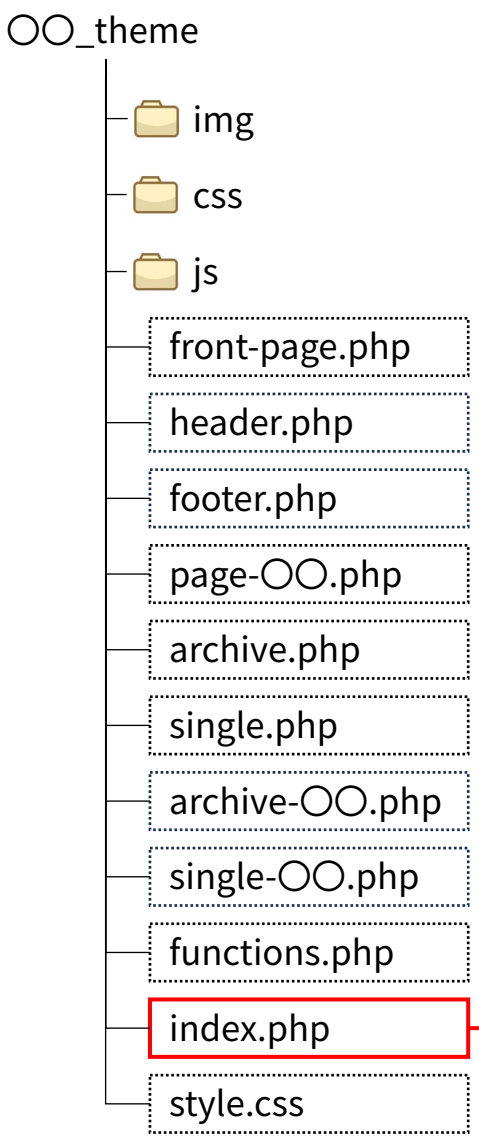

## functions.php

functions.phpとは、読み込むcssやjsファイルの設定、機能の拡張など、デフォルトの動作を変更するためのファイルです。
jsファイルを追加した場合は、ここに読み込み処理を書く必要があります。

ここにはその名の通り、多数の関数が記述されています。
テーマの作成作業の際、functions.phpは最適な状態で設定されていることが多く、ここを触ると正常に動かなくなる事もあります。

テーマの中枢となるファイルなので、ここを触る場合は必ず他の人に報告、確認しましょう。






## index.php

**WordPressがテーマを認識するために必要なファイルが index.php です。**

**基本的には何も記述されていないことが多いですが、万が一固定ページのファイルがないなどの理由で、何も表示されない時に表示されるのが index.php です。**

**その時に何も表示されないことがないように、内容を記述することもあります。**



Re start

# テーマを表示しよう

## ① Localでサイトを作成する

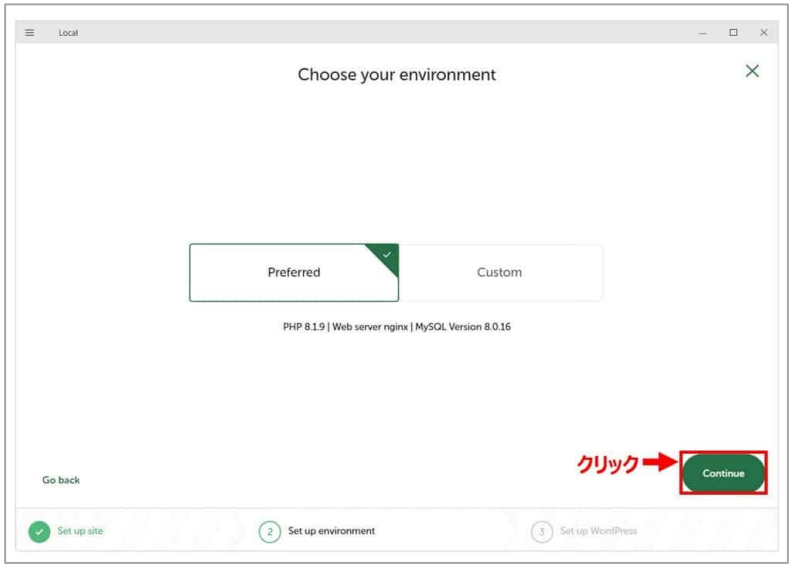
「 create a new site 」
を選択して、continueを押す

## ② Webサイト名を設定する

識別しやすい名前を入力して
Continue を押す

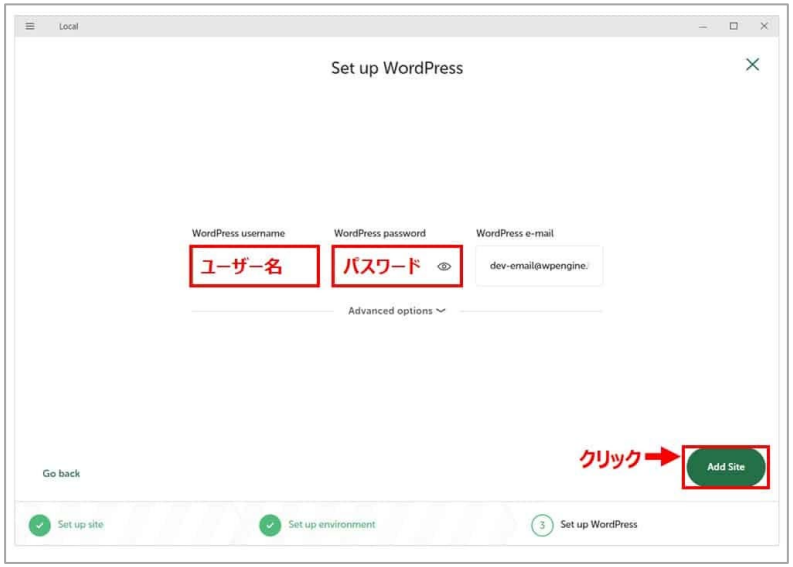
# テーマを表示しよう

## ③ Preferred を選択




Preferred を選択して Continueを押す。

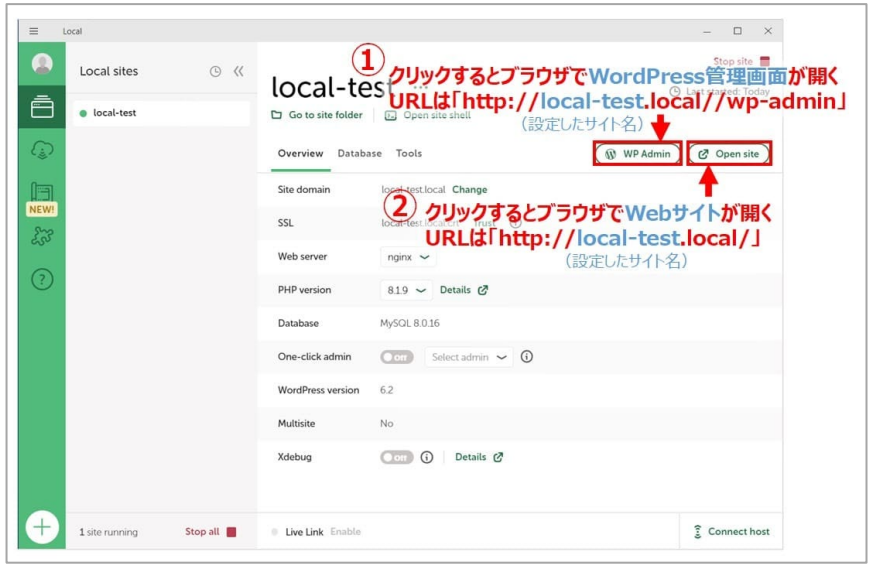
## ④ ユーザー情報を設定




WordPressの管理画面へのログイン情報を入力する。
（両方とも admin と設定するのが一般的です）

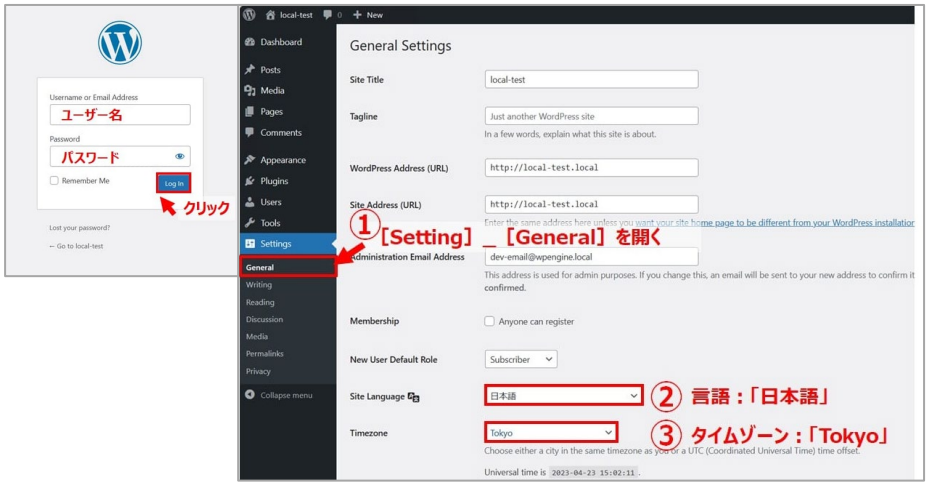
# テーマを表示しよう

## ⑤ サイト・管理画面を開く




「WP Admin」を押すとWordPressの管理画面が開く。
「Open site」を押すとWebサイトが開く。

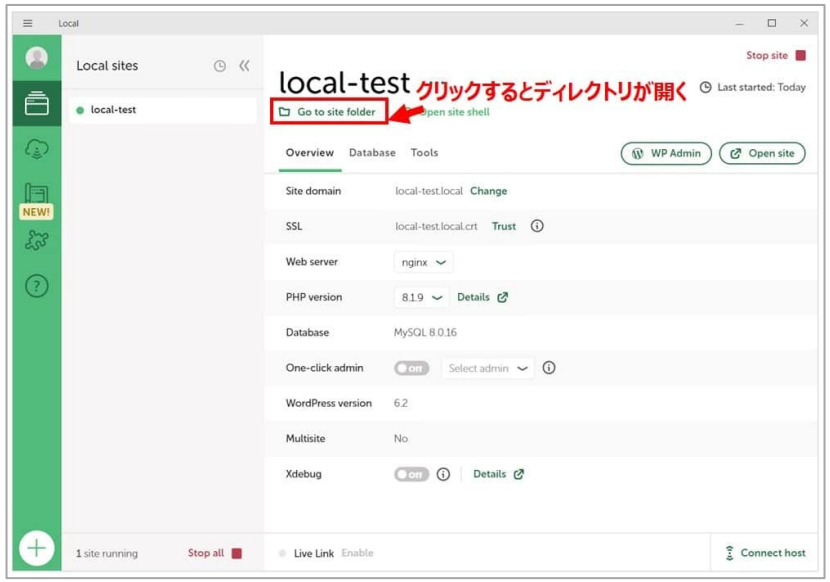
## ⑥ 管理画面にログイン




入力したユーザー情報を入力してログイン。
Setting→General から言語とタイムゾーンを日本に変更。
※Localの設定で、日本語化作業は必要なくなります。

# テーマを表示しよう

## ⑦ インストールフォルダを確認する




「Site folder」を押してWordPressが
インストールされているフォルダを開く。

## ⑧ フォルダにテーマを格納

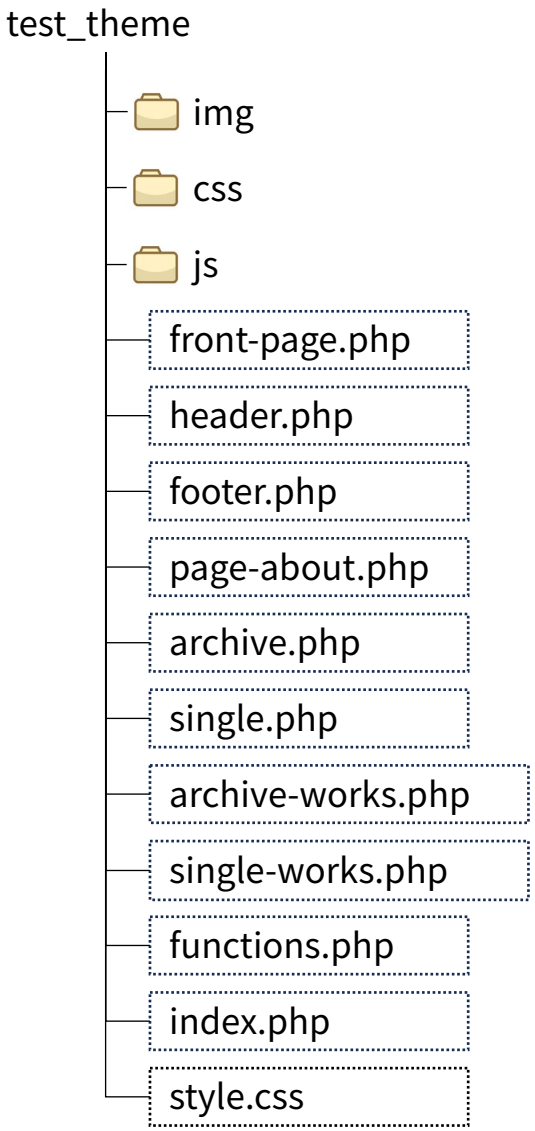
サイトのフォルダが開く。

app > public > wp-content > themes

ここにテーマフォルダをまるごと格納。
管理画面の

外観 > テーマ

で、テーマが表示されていたら成功。
テーマを有効化して、表示してみましょう。




## 配布したテーマの構造

一般的な企業のホームページを作る想定でテーマをつくっていきます。

最終的なサイト構成は
- TOP
- 会社概要
- 制作実績
- お知らせ
- お問い合わせ

です。

## テンプレートファイルの内容

```
<?php get_header();?>
<main class="main">
  <div class="eye_catch">
    <div class="car_anim_wrap">
      <video class="pc_only car_video" autoplay muted loop p
      <video class="sp_only car_video" autoplay muted loop p
    </div>
    <div class="catch_wrap">
      <img class="catch_img" src="<?php echo get_template_di
      <p class="counsel_text ">お気軽にご相談ください</p>
      <img class="catch_tel_img" src="<?php echo get_templat
    </div>
  </div>
  <!-- サービスメニューセクション -->
  <section class="service_menu">
    <h1 class="service_heading">
      電気と住居のお困りごとは<br />でんき住設にお任せください！
    </h1>
    <div class="service_menu_container">
      <div class="service_menu_box">
        <div class="service_menu_image">
          <img src="<?php echo get_template_directory_uri()
        </div>
        <p class="service_menu_text">住宅設備・修理</p>
      </div>
      <div class="service_menu_box">
        <div class="service_menu_image">
          <img src="<?php echo get_template_directory_uri()
        </div>
        <p class="service_menu_text solar_txt">
          太陽光発電・蓄電池・<br />
          オール電化
        </p>
      </div>
      <div class="service_menu_box">
        <div class="service_menu_image">
```

Wordpessのテーマ構造通りにファイルを作成し、テーマを適用することができました。

では、front-page.php や page-〇〇.php などのテンプレートファイルには何を記述したらいいのでしょう？

左にあるのは、実際のサイトの front-page.phpの記述です。見ての通り、通常のHTMLが記述されていますね。

WordPressのテンプレートファイルは、HTMLで組む場合の body 部分を記述すればそのまま表示されます。

ですが、一部リンクや画像などは決まった記述をしないと正常に動作しなかったり、表示されなかったりします。

また、ヘッダーやフッターは別で設定されているので、呼び出しの処理を記述する必要があります。



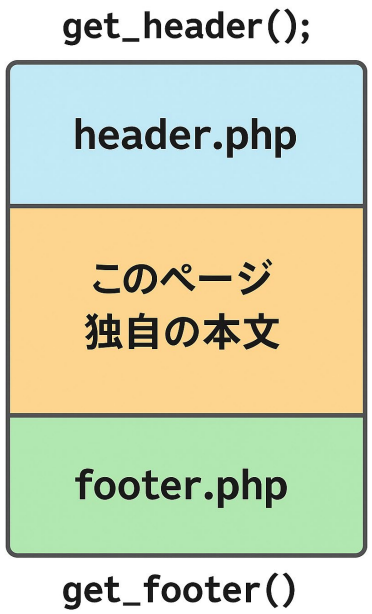
Re start

## ヘッダー・フッターの呼び出し

テンプレートファイルでヘッダー・フッターを呼び出す際は、以下のように記述します。

```
<?php get_header(); ?>
このページの本文
<?php get_footer();?>
```

get_header( ) で header.php 、get_footer( )で footer.php に記述されている内容を呼び出すことができます。



## header.phpの記述

header.phpにはHTML基本構造の最上部から記述していきます。
<header>以外にも、各ページ共通で存在している要素などを記述することもあります。
body、htmlの閉じタグは不要です。

```
<!DOCTYPE html>
<html lang="ja">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>初めてのテーマづくり</title>
</head>
<body>
     <header class=“header”>
         <div class="header_logo">
             <a href=“/”>Test株式会社</a>
         </div>
         <nav class="header_nav">
             <ul class="header_nav_list">
                 <li class="header_nav_item"><a href=“/”>TOP</a></li>
                 <li class="header_nav_item"><a href=“/about/”>会社概要</a></li>
                 <li class="header_nav_item"><a href=“/works/”>制作実績</a></li>
                 <li class="header_nav_item"><a href=“/news/”>お知らせ</a></li>
                 <li class="header_nav_item"><a href=“/contact/”>お問い合わせ</a></li>
             </ul>
         </nav>
     </header>
```

## footer.phpの記述

footer.phpには<footer>と、body、htmlの閉じタグを記述します。

```
    <footer class="footer">
        <small>&copy; Test株式会社 All Rights Reserved.</small>
    </footer>

</body>
</html>
```



Re start

## 各テンプレートファイルでの記述

まず、front-page.phpに記述をしていきます。現在はHTMLだけ記述してあります。

front-page.php

```
<!-- ここに本文を記載 -->
<div class="main_visual">
  <img src="img/fv.jpg" alt="">
</div>

<section>
  <h2 class="sec_title">お知らせ</h2>
  <a href="" class="news_item">
      <img src="img/news.jpg" alt="">
      <div class="news_info">
        <div class="news_meta">
          <time class="news_date">2023.10.01</time>
          <p class="news_category">お知らせ</p>
        </div>
        <h3 class="news_title">お知らせタイトルが入ります。</h3>
      </div>
  </a>
</section>
<!-- ここまで本文を記載 -->
```

## サイトの表示

管理画面の左上にあるホームアイコンをクリックするとサイトのTOPを確認できます。
管理画面とサイトの画面は頻繁に行き来するため、別タブで開いておくとよいでしょう。

## ヘッダーとフッターの読み込み

各テンプレートでヘッダーとフッターを読み込みます。

```
<?php get_header(); ?>

<!-- ここに本文を記載 -->
```

```
<!-- ここまで本文を記載 -->

<?php get_footer();?>
```

記述後、画面を更新してみましょう。



Re start

## 画像の表示

Wordpressでは、相対パスのままだと画像の場所がうまく指定できません。

```
<!-- ここに本文を記載 -->
<div class="main_visual">
  <img src="img/fv.jpg" alt="">
</div>
```

この記載だと、http://〇〇.local/img/fv.jpg　に画像を見に行きます。
実際に画像があるのは、wp-content -> themes -> test_theme -> img の中です。

正しい場所を指定するためにテーマフォルダ内のファイルのURLを取得する関数を使います。
書き終えたら、更新して検証モードでパスを確認してみましょう。

```
<img src="<?php echo get_theme_file_uri('/img/fv.jpg'); ?>" alt="">
```



Re start

## cssファイルからの画像指定

パスに特別な記載をするのはテンプレートファイル内のみです。
例えばcssで、背景画像を設定する場合などは
記載するcssファイルからimgフォルダまでの相対パスで指定できます。

front-page.css

```
.news_section{
    background-image: url('../img/bg.jpg');
}
```



Re start

## 別のページへのリンク設定

これまで複数ページ作成する場合には、ページの数だけhtmlファイルを作成していました。
（相互リンクは相対パス）

しかし、今作成しているのはテンプレートファイル（着せ替えテーマ）ですので
リンクの指定にも関数を使用します。
header.phpを書き換えてみましょう。

```
<li class="header_nav_item"><a href="<?php echo esc_url(home_url('/')); ?>">TOP</a></li>
<li class="header_nav_item"><a href="<?php echo esc_url(home_url('/about/')); ?>">会社概要</a></li>
```

home_url(‘/’) 関数は、「サイトのURLを取得する関数」です。

※esc_url() 関数は、「URLを安全に表示できるように整えてくれる関数」です。



Re start

# 固定ページの作成

① 管理画面サイドメニュー「固定ページ」から新規作成。

② 記事のタイトルを入力。
ここでは「会社概要」と入れます。

# 固定ページの作成

## ③固定ページタブの「スラッグ」をクリック

## ④「about」と入力し、公開ボタンを押す。

これで/aboutページが完成しました。

## 固定ページテンプレートの編集

先ほど管理画面から、URLが「/about/」となる固定ページを作成しました。
そのページの表示に使用されるテンプレートファイルは
「page-about.php」となります。

固定ページには会社概要の表が書いてあります。
ヘッダーのリンクから/about/ページに遷移してみましょう。

表示出来たら、同じ手順でお問い合わせ(contact)ページも作成しましょう。

## cssの読み込み

cssの読み込み処理は、これまで通り<head>タグ内に記述することもできますが
Wordpress自体の持つCSSとの競合回避や、読み込み順の制御が可能になるなど
メンテナンス性を高くするためfunctions.phpで記述することが推奨されています。

functions.php

```
<?php

/*====================================
 * CSS読み込み
====================================*/
function enqueue_style(){

    wp_enqueue_style('reset', get_template_directory_uri() . '/css/reset.css');

}
```

```
wp_enqueue_style( $handle, $src, $deps, $ver, $media );
```

第一引数…スタイルの名前（id)。
第二引数…スタイルシートまでのパス。
（第三、第四、第五引数は省略可）



Re start

## 名前の重複について

スタイルの名前が重複すると、２つ目以降は読み込まれません。

functions.php

```
function enqueue_style(){

    wp_enqueue_style('reset', get_template_directory_uri() . '/css/reset.css’);
    wp_enqueue_style('reset', get_template_directory_uri() . '/css/reset1.css’);
    wp_enqueue_style('reset', get_template_directory_uri() . '/css/reset2.css');
}
```

また、wordpressに最初から存在している名前と重複しても読み込まれません。
例）about

functions.php

```
function enqueue_style(){

    wp_enqueue_style('about', get_template_directory_uri() . '/css/reset.css’);
}
```



Re start

## cssの読み込み関数の呼び出し

関数は定義しただけでは、処理が動いてくれません。
フックという仕組みを使って、どのタイミングでどの関数を呼ぶのかを指定します。

functions.php

```
function enqueue_style(){
}


add_action('wp_enqueue_scripts', 'enqueue_style');
```

```
// 基本的な使い方
add_action( 'フックの名前', '実行したい関数名' );
```



Re start

## headタグへの記述

最後にheader.phpのheadタグ内でCSS読み込みの記述が出力されるように記述します。
headタグ内の一番最後に、wp_head()関数を追記しましょう。
記述したら画面を更新し、CSSが読み込まれていることを確認しましょう。

header.php

```
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>初めてのテーマづくり</title>
    <?php wp_head(); ?>
</head>
```

※functionsで定義したCSS以外にも、wordpressのデフォルトCSSなども読み込まれます。
忘れずに記述しましょう。



Re start

## cssの読み込み

「style.css」「header.css」「footer.css」「 about.css」「front-page.css」も読み込んでみましょう

functions.php

```
function enqueue_style(){
  wp_enqueue_style('reset', get_template_directory_uri() . '/css/reset.css’);
  //追加のCSSをここに記述します。
}
add_action('wp_enqueue_scripts', 'enqueue_style');
```



Re start

# jsの読み込み

## jsの読み込み

```
wp_enqueue_script( $handle, $src, $deps, $ver, $in_footer );
```

| | |
|---|---|
| $handle | スクリプトの識別名 |
| $src | スクリプトのパス |
| $deps | 依存スクリプト |
| $ver | スクリプトのバージョン |
| $in_footer | スクリプトの読み込み位置（trueで/body前、falseで/head前） |

functions.php

```
function enqueue_script(){

  // wp_enqueue_script( $handle, $src, $deps, $ver, $in_footer );
  wp_enqueue_script('main', get_template_directory_uri() . '/js/main.js', array(), '1.0.0', true);
}
add_action('wp_enqueue_scripts', 'enqueue_script');
```



Re start

## jsの読み込み

基本的にjsはbody閉じタグの前で読み込むので、第五引数はtrueにします。
最後に定義したjsが読み込まれるよう、footer.phpのbody閉じタグの上にwp_footer()を記述します。

footer.php

```
    <footer class="footer">
        <small>&copy; Test株式会社 All Rights Reserved.</small>
    </footer>
    <?php wp_footer(); ?>
</body>
</html>
```

コンソールにmain.jsの処理が出ているか確認してみましょう。



Re start

## WordpressでのjQuery

**Wordpressには、初期状態でjQueryが存在しています。**
**functions.phpに定義することで使用可能です。**

footer.php

```
function enqueue_script(){

    wp_enqueue_script('jquery');//wordpressに同梱されているjQueryを読み込む
}
```



Re start

## WordpressのjQueryを使用する場合の注意点

### ①$を使うとエラーになる

```
JQMIGRATE: Migrate is installed,        jquery-migrate.min.js?ver=3.3.2:2
version 3.3.2
▸Uncaught TypeError: $ is not a function                    main.js:8
    at main.js:8:1
```

### ■WordPressのjQueryを使用する場合の書き方の例

```
jQuery(function () {
    jQuery('.sample').css('color', 'red');
    jQuery('#sample').css('color', 'white');
});
```

```
jQuery(function($) {
    $('.sample').css('color', 'red');
    $('#sample').css('color', 'white');
});
```



Re start

## WordpressのjQueryを使用する場合の注意点

### ②jQuery依存のスクリプトは読み込む順番に注意する

第三引数には、先に読み込む必要があるjsのハンドルを指定します。

```
wp_enqueue_script('jquery');//wordpressに同梱されているjQuery
wp_enqueue_script('main', get_template_directory_uri() . '/js/main.js', array('jquery'), '1.0.0', true);
```

### ③WordpressのjQueryを使いたくない場合

```
wp_deregister_script('jquery');
wp_register_script('jquery', 'https://code.jquery.com/jquery-3.6.0.min.js', array(), '3.6.0', true);
```

上記のように記述すれば無効化できるが、お問い合わせフォームのプラグインなどは標準のjQueryを使用しているため、**不具合の原因になる**ことがあります。



Re start

## 投稿一覧ページ（アーカイブ）を表示する

**固定ページは管理画面で新規作成が必要でした。**
**投稿の一覧ページはデフォルトの状態では「無し」のため、functions.phpでの変更が必要です。**
**今回はすでに記述しています。**

```
/*=====================================
 * 投稿のアーカイブページを作成
=====================================*/
function set_post_archive($args, $post_type){ // 設定後に（パーマリンク更新すること）
    if ('post' == $post_type) {
        $args['rewrite'] = array('with_front' => false);
        $args['has_archive'] = 'news';
        $args['label'] = 'お知らせ';
    }
    return $args;
}
add_filter('register_post_type_args', 'set_post_archive', 10, 2);
```

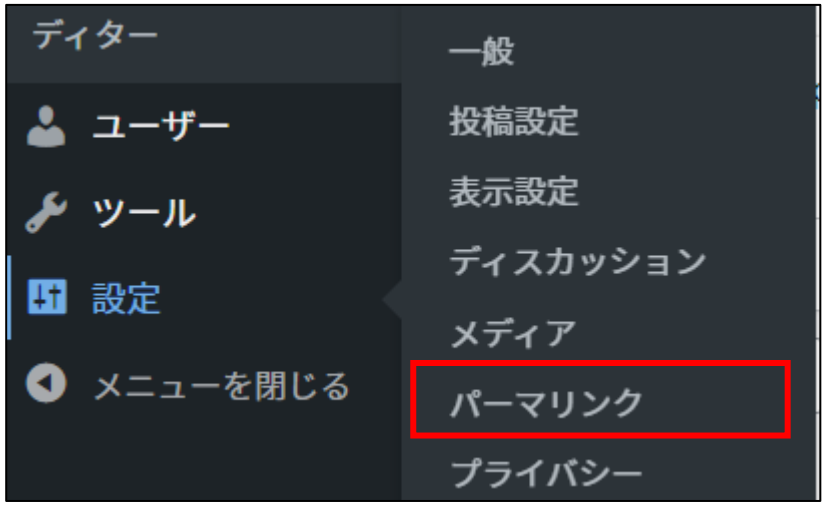
has_archiveで指定した文字列が一覧ページのURLになります。
labelは,管理画面での表示名です。



Re start

## 投稿一覧ページ（アーカイブ）を表示する

ページの定義をfunctions.phpなどで記述して変更した際には、
Wordpress内でサイトのURL構成を再構築する必要があります。



何も変更せず「変更を保存」を押します。
/news/ にアクセスしてみましょう

## 記事を表示する

HTMLで組んだ記事一つ分の要素グループを記事の数があるだけ繰り返す処理をいれます。

### 記事の開始位置

```
<?php if ( have_posts() ) : ?>
  <?php while ( have_posts() ) : the_post(); ?>
  <!-- ここから記事 -->
```

### 記事の修了位置

```
    <!-- ここまで記事 -->
    <?php endwhile;?>
<?php endif;?>
```



Re start

# 記事を投稿する

①サイドメニューの「お知らせ」をクリック。「投稿を追加」ボタンを押します。

②「タイトル」と「本文」を入力して「公開」ボタンを押します。

Re start

## 一覧ページを確認する

2023.10.01 田中太郎

**お知らせタイトルが入ります。**

お知らせ

2023.10.01 田中太郎

**お知らせタイトルが入ります。**

お知らせ

**投稿を追加すると表示数が増えます。**



Re start

## 記事の情報を取得・表示する

投稿は、「タイトル」や「本文」、「投稿日」など様々な情報を持っています。
それらをループの中で取得する関数を使って表示します。

◆タイトルを表示する

```
<h3 class="news_title"><?php the_title(); ?></h3>
```

◆タイトルを取得する

```
<?php $title = get_the_title(); ?>
<h3 class="news_title"><?php echo $title; ?></h3>
```

the_○○() という関数は画面側に表示する機能も含んでいます。
get_the_○○() という関数は値を取得するだけなので、取得したうえでその値を加工する必要がある場合などに使用します。最終的に画面に表示するには　「echo」をつけます

## 記事の情報を取得・表示する

### ◆投稿日を表示する

```
<time class="news_date" datetime="2025-4-27"><?php the_date(); ?></time>
```

↑引数に何も指定しない場合、管理画面で設定した形式になります。

```
<?php the_date('Y.n.j'); ?>
```

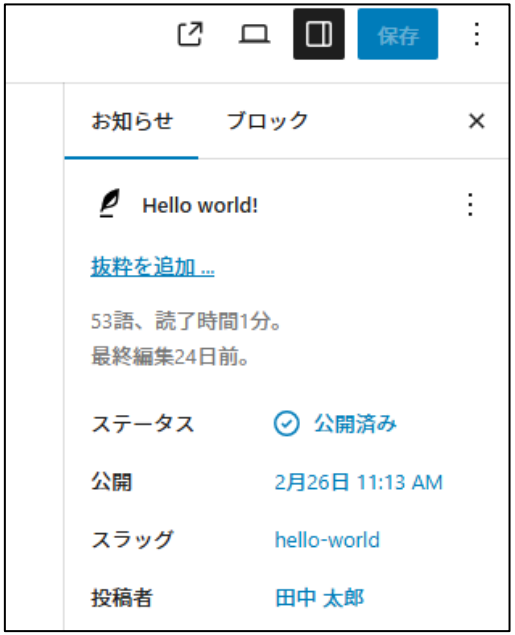
### ◆日付のフォーマット例

| フォーマット | 表示例 | 説明 |
|---|---|---|
| 'Y年n月j日' | 2025年5月3日 | 和風で定番な日付表示 |
| 'Y/m/d' | 2025/05/03 | 管理画面向けなど |
| 'Y年n月j日 (D)' | 2025年5月3日 (Sat) | 曜日付き |
| 'Y年n月j日 G:i' | 2025年5月3日 9:07 | 時間まで表示 |
| 'F j, Y' | May 3, 2025 | 海外サイトっぽい英語表記 |
| 'Y.m.d H:i:s' | 2025.05.03 09:07:59 | ログやシステム情報向き |

## 記事の情報を取得・表示する

### ◆アイキャッチ（サムネイル画像）を有効化する

記事には、アイキャッチと呼ばれる画像を設定することができます。この画像は一覧ページなどで記事の内容を表すために用いるものです。
まずは管理画面からアイキャッチを設定できるように、functions.phpに記述していきましょう。




初期状態ではアイキャッチ登録機能が有効になっていません

（←記事の編集画面右側）

## 記事の情報を取得・表示する

functions.php

```
/*=====================================
 * サムネイルを有効にする
=====================================*/
add_theme_support('post-thumbnails');
```

記述したら、記事の編集画面を開きます。

「アイキャッチ画像を設定」が出ていたらOKです。
クリックして画像を選んで設定しましょう。

## 記事の情報を取得・表示する

まずは、archive.phpで画像タグの上にアイキャッチを出してみましょう。

```
<?php the_post_thumbnail(); ?>
<img src="<?php echo get_theme_file_uri('/img/news.jpg'); ?>" alt="" class="news_img">
```

アイキャッチを設定した記事には画像が表示されます。
しかし、この関数で表示されるimgタグにはclassがついていないため
大きさなどが制限できていない状態です。

```
<?php the_post_thumbnail( 'thumbnail', ['class' => 'news_img'] ); ?>
```

このように記述することで、imgタグにクラスがつきます。



Re start

## 記事の情報を取得・表示する

もともとあったimgタグを削除してしまうと、アイキャッチがない記事のときに見た目が崩れてしまいます。
多くの場合、アイキャッチ画像がない場合はロゴを表示することが多いので
アイキャッチがある場合、ない場合の判定をして分岐させてみましょう。

```
has_post_thumbnail()
```

この関数は、アイキャッチがある場合にはtrue（はい）、ない場合はfalse(いいえ)を返します

```
<?php if(has_post_thumbnail()): ?>
      <?php the_post_thumbnail( 'thumbnail', ['class' => 'news_img'] ); ?>
<?php else: ?>
      <img src="<?php echo get_theme_file_uri('/img/news.jpg'); ?>" alt="" class="news_img">
<?php endif; ?>
```

## 詳細ページへのリンク設定

一覧ページから記事一つ一つのページに飛ぶためのURLを取得して設定しましょう

```
<a href="<?php the_permalink(); ?>" class="news_item">
```

詳細ページのURLの形式は、wordpressの管理画面から変更することができます。
初期状態では「/投稿名」になっています。
「設定」＞「パーマリンク」から変更できます。

## 詳細ページ（single.php）の表示

さきほど、一覧ページで詳細へのリンクを設定したのでそこから詳細ページに飛びます。

①まずは、CSSを適用しましょう。

記事の情報を表示する処理については、一覧ページと同じ手順です。

②一覧ページを参考にしながら、
・タイトル
・日付
・投稿者名
を表示してみましょう



Re start

## 記事の本文を取得する

本文をいれるためのdivを作っておきましょう。
今回は をつくっています。

```
<div class="news_content">
   <?php the_content(); ?>
</div>
```

これで本文が取得できます。本文には文章（pタグ）や見出し(h2タグ）が含まれます。
取得した本文にCSSを適用させたいときには、ラッパークラス（今回はnews_content）の中のh2タグやpタグに対して記述します。

基本的にタグ名に直接CSSを書かないですが、取得した本文に関しては例外です。

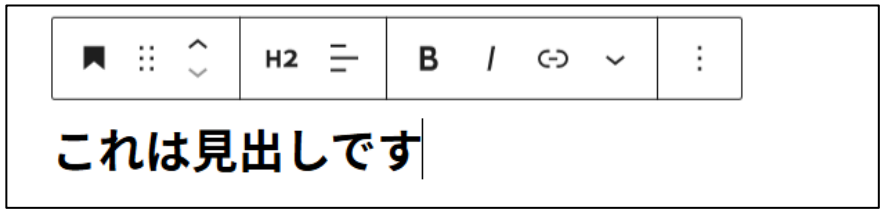
## 本文をさらに追加する

①サイドメニューの「お知らせ」をクリック。
記事のどれかを編集します。

②黒い＋ボタンをクリックすると様々なブロックを選択できます。
まずは見出しを選択してみましょう。

## 本文をさらに追加する

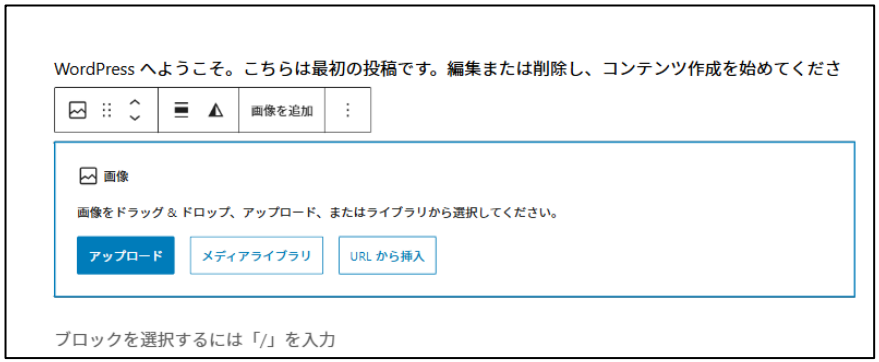
③見出しを入力できます。H2以外にも
H3~H6まで選択できます。
（※H1も選択できますが、ブログタイトルが
H1のためH2がデフォルトになっています）




④今度は画像ブロックを追加してみます。

## 本文をさらに追加する

⑤「アップロード」「メディアライブラリ」「URL」からの３つから画像指定方法を選びます。




⑥挿入したい画像を選んだら右下の挿入をクリックします。
本文を更新したら保存をして、詳細ページをリロードしてみましょう。

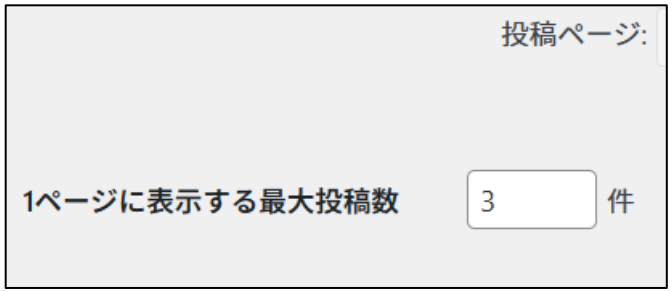
## ページネーションの作成

Webサイトのページを分割して表示し、ユーザーが内容を読みやすくするために、複数のページに分ける機能のことをページネーションといいます。

記事の一覧（archive.php）で記事の数が増えてきたとき、１ページに収まる件数を超えたらページネーションを表示させることができます。

```
<!-- ページネーション -->
<?php the_posts_pagination(); ?>
```

記述したら、管理画面の「設定」＞「表示設定」の１ページに表示する最大投稿数を少ない数字に変更し、記事を追加してページネーションの表示を確かめてみましょう。



## ページネーションの作成

ページネーションの表示関数にも、様々なオプションが用意されています。

```
$args = array(
        'mid_size' => 1,
        'prev_text' => '前のページ',
        'next_text' => '次のページ',
        'screen_reader_text' => ' ',
);
the_posts_pagination($args); ?>
```

出力されるHTMLはこのような構造になっています。

```
▼<nav class="navigation pagination" aria-label=" "> == $0
  <h2 class="screen-reader-text"> </h2>
  ▼<div class="nav-links">
    <span aria-current="page" class="page-numbers current">1</span>
    <a class="page-numbers" href="http://test.local/news/page/2/">2</a>
    <a class="next page-numbers" href="http://test.local/news/page/2/">次のページ</a>
  </div>
</nav>
```



Re start

## ページネーションの作成

**出力されるHTML構造にたいしてcssを設定していきましょう。**
**特に、現在選んでいるページがわかるように現在のページ番号の要素に対しては**
**違う色の指定があることがほとんどです。**

| クラス名 | スタイル |
|---|---|
| .pagination | ページネーション全体 |
| .nav-links | ページネーションのリンク |
| .page-numbers | 各ページ番号 |
| .page-numbers.current | 現在のページ番号 |
| .page-numbers.prev,<br>.page-numbers.next | 「前のページ」と「次のページ」リンク |
| .page-numbers.dots | 省略記号（…） |



Re start

## ページネーションの作成

下の画像のようなページネーションになるようにarchive.cssに記述してみましょう

## ページネーションの作成

HTML構造をあらかじめ作っている場合や、もっと自由にページネーションをカスタムしたい場合にはpaginate_links()という関数を使用します。

```
<?php echo paginate_links(); ?>
```

見た目に若干の違いはありますが、出力されるHTMLを見るとクラス名はほぼ同じです。

```
<a class="prev page-numbers" href="http://pagination-test.local/posts/">« 前へ</a>
<a class="page-numbers" href="http://pagination-test.local/posts/">1</a>
<span aria-current="page" class="page-numbers current">2</span>
<a class="page-numbers" href="http://pagination-test.local/posts/page/3/">3</a>
<a class="page-numbers" href="http://pagination-test.local/posts/page/4/">4</a>
<span class="page-numbers dots">…</span>
<a class="page-numbers" href="http://pagination-test.local/posts/page/6/">6</a>
<a class="next page-numbers" href="http://pagination-test.local/posts/page/3/">次へ »</a>
```

## ページネーションの作成

**paginate_links()関数だけにあるオプションに、typeがあります。**
**aタグやspanタグで構成されたHTMLを配列で取得することでより柔軟なカスタマイズが可能です。**

```
<div class="my-pagination"><ul>
<?php
    $args = array(
        'mid_size' => 1,
        'type' => 'array'
    );
    $pagination = paginate_links($args);
    foreach ($pagination as $page) {
        // currentページにはクラスを付ける
        if (strpos($page, 'current') !== false) {
            echo '<li class="current">' . $page . '</li>';
        } else {
            echo '<li>' . $page . '</li>';
        }
    }
?>
</ul></div>
```



Re start

## カテゴリーの作成

記事をジャンルによって絞り込む機能は、多くのサイトについています。
まずは、記事につけるためのカテゴリーを作成しましょう。

# カテゴリーの作成

①サイドメニューの「お知らせ」＞「カテゴリー」をクリックします。

②「名前」は実際にホームページ上にでるカテゴリー名をいれます。
スラッグはURLの文字になるので、半角英数字、区切りは-(ハイフン)を使用します。

## カテゴリーの作成

③記事の編集画面を開きます。編集画面の右下にカテゴリーのチェックがあるので、付与したいカテゴリーをクリックして保存します。

## 記事に付与されたカテゴリーを取得・表示する

**記事に割り振られたカテゴリーを取得・表示する関数を2つ紹介します。**
**リスト形式で表示まですることができるもの、取得のみ行い必要なデータのみ表示するものがあります。**

| | |
|---|---|
| the_category() | カテゴリーをリスト形式で表示 |
| get_categories() | カテゴリーをオブジェクト配列で取得 |

**the_category() の実行結果は下記のようになる。aタグもついてしまうので**
**記事に付与されたカテゴリから、カテゴリに飛びたい場合は使用可能です。**

```
<ul class="post-categories">
    <li><a href="カテゴリーアーカイブページのURL" rel="category tag">カテゴリー名〇〇</a></li>
    <li><a href="カテゴリーアーカイブページのURL" rel="category tag">カテゴリー名△△</a></li>
</ul>
```



Re start

## 記事に付与されたカテゴリーの取得

archive.phpを編集します。
記事の取得ループの中の、「記事についたカテゴリー」の箇所を
下記のように書き換えましょう。カテゴリーは複数付与されている場合もあるので、ループで表示します。

```
<!-- 記事についたカテゴリー -->
<ul class="news_category">
    <?php
    $categories = get_the_category();
    foreach ( $categories as $category ) {
        echo '<li">' . esc_html( $category->name ) . '</li>';
    }
    ?>
</ul>
```

## 全てのカテゴリーを取得・表示する

**記事の一覧ページで、特定のカテゴリーがついた記事だけに絞り込むためのボタンを実装してみましょう。全てのカテゴリーを取得する関数を２つ紹介します。**

| | |
|---|---|
| wp_list_categories() | カテゴリーをリスト形式で表示 |
| get_categories() | カテゴリーをオブジェクト配列で取得 |

## wp_list_categories()の使い方

カテゴリーによる絞り込みの箇所に下記のように記載しましょう。

```
<!-- カテゴリーによる絞り込み -->
<ul class="category_list">
    <?php
    wp_list_categories(array(
        'title_li' => ''
    ));
    ?>
</ul>
```

記述すると、liタグの中にaタグのHTML構造が表示されます。
カテゴリーで絞り込んだページへのリンクも入っているので、手軽にリンク付きのカテゴリー一覧を表示することができます。
また、いずれかのカテゴリーのリンク先に飛ぶと、そのカテゴリーのliタグにcurrent-catというクラスが自動的につきます。
選択中を示す場合にはこのクラスを使うことができます。



Re start

## get_categories()の使い方

前ページの記述の下に、下記のように記述しましょう。

```
<ul class="category_list">
    <?php
    $categories = get_categories();
    foreach ( $categories as $category ) {
        echo '<li><a class="category_link" href="' . esc_url( get_category_link( $category ) ) . '">' .
esc_html( $category->name ) . '</a></li>';
    }
    ?>
</ul>
```

get_categories()は、カテゴリーを配列で返すだけの関数です。
そのため、liタグに任意のクラスをつけるなど、HTML構造は自由にカスタムすることができます。
カテゴリーアーカイブへのURL取得や、選択中のカテゴリ判定は自分で行わなければなりません。



Re start

## 現在選択中のカテゴリーにクラス付与する

get_queried_object_id()という関数は、今表示しているページで「クエリされたオブジェクト（投稿・カテゴリ・固定ページなど）」の ID を取得することができます。

カテゴリーアーカイブページ（カテゴリで絞り込んだ状態）では、絞り込んだカテゴリのIDが取得できるため、ループの中でIDを比較することで、選択中のカテゴリにクラス付与できます。

```
$current_cat_id = get_queried_object_id();//現在のカテゴリーIDを取得
$categories = get_categories();
foreach ( $categories as $category ) {
    $class = ($category->term_id == $current_cat_id) ? 'current-category' : ‘’;
    echo '<li class="' . $class .'"><a class="category_link" href="' . esc_url( get_category_link( $category) ) .
'">' . esc_html( $category->name ) . '</a></li>';
}
```



Re start

## TOPのお知らせフィード

**TOPページには、お知らせ記事の最新数件を表示することがよくあります。**
**archive.phpと同じように記事の開始と終了を囲い、記事情報を表示しましょう。**

```
<?php if ( have_posts() ) : ?>
        <?php while ( have_posts() ) : the_post(); ?>
        <a href="" class="news_item">
            <img src="<?php echo get_theme_file_uri('/img/news.jpg'); ?>" alt="" class="news_img">
            <div class="news_info">
                <div class="news_meta">
                    <time class="news_date">2023.10.01</time>
                    <p class="author">田中太郎</p>
                </div>
                <h3 class="news_title">お知らせタイトルが入ります。</h3>
                <ul class="news_category">
                    <li>お知らせ</li>
                </ul>
            </div>
        </a>
        <?php endwhile;?>
    <?php endif;?>
```

## 投稿タイプについて

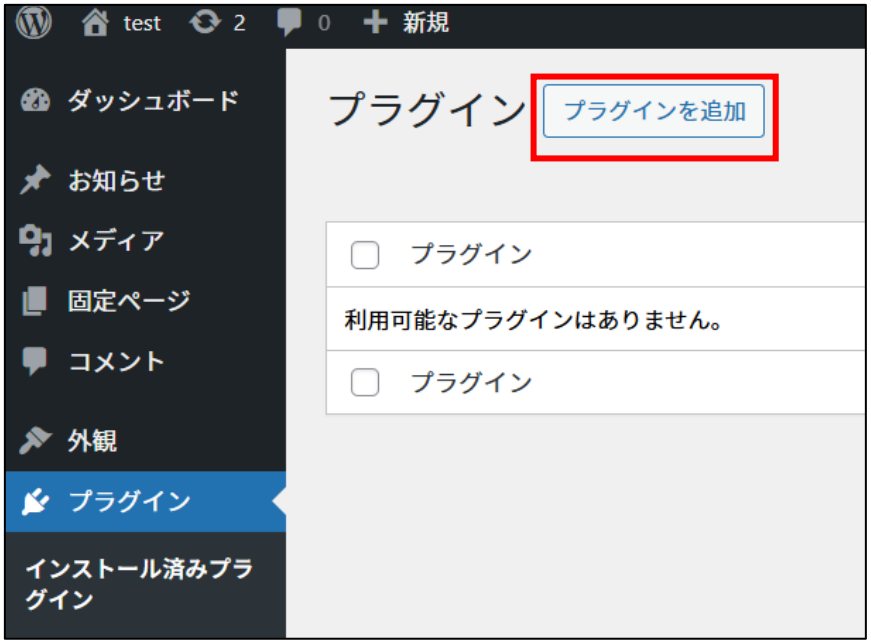
**Wordpressには標準でいくつかの投稿タイプが用意されています。投稿タイプは「コンテンツの種類」のことです。**

| コンテンツ例 | 内容 |
|---|---|
| 投稿（post） | ブログ記事・ニュースなどの時系列情報 |
| 固定ページ（page） | お問い合わせ・会社概要などの独立したページ |
| メディア（attachment） | 画像やPDFなどアップロードされたファイル |

**ブログ以外にも、投稿コンテンツを増やしたい場合**
**（制作実績・スタッフ紹介など）**
**Wordpressでは自分で投稿タイプを増やすことができます。**
**自分で増やした投稿タイプのことを「カスタム投稿タイプ」と呼びます。**

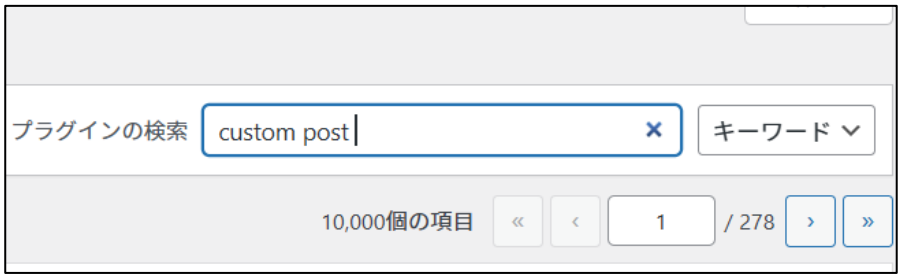
## プラグインの追加

**投稿タイプの追加には、プラグインを使うのが便利です。プラグインは、インストールすることで簡単にさまざまな機能を追加することができます。**



## カスタム投稿タイプの追加

①右上のプラグインの検索から「Custom Post Type UI」を検索します。

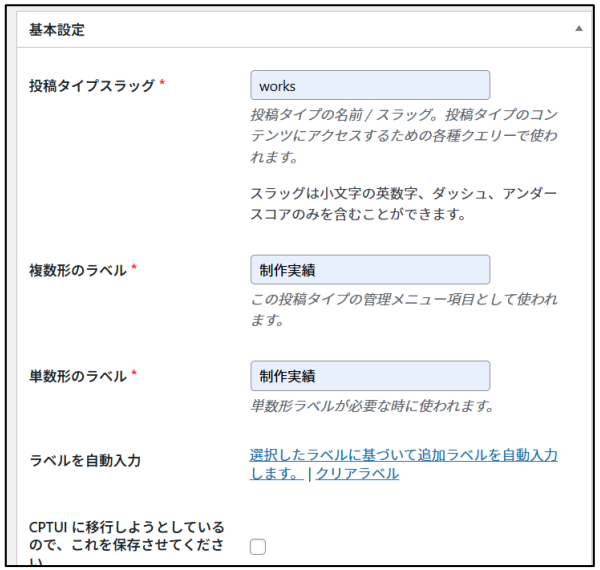



②「今すぐインストール」を押してインストール、その後表示される「有効化」を押します。

## カスタム投稿タイプの追加

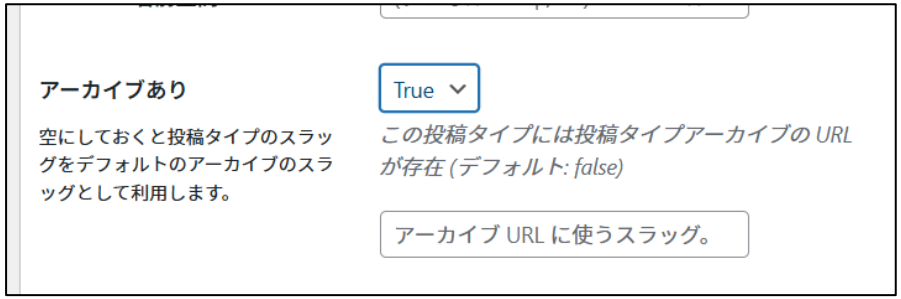
③サイドメニューに「CPT UI」というメニューが増えるので、投稿タイプの追加と編集を選択しましょう。

④投稿タイプスラッグは英語、複数形と単数形のラベルは管理画面での表示名を入力します。

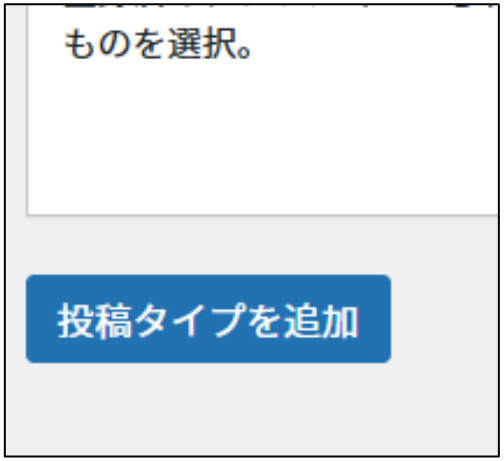


## カスタム投稿タイプの追加

⑤下にスクロールし、設定項目の「アーカイブあり」の項目を「True」に変更します。

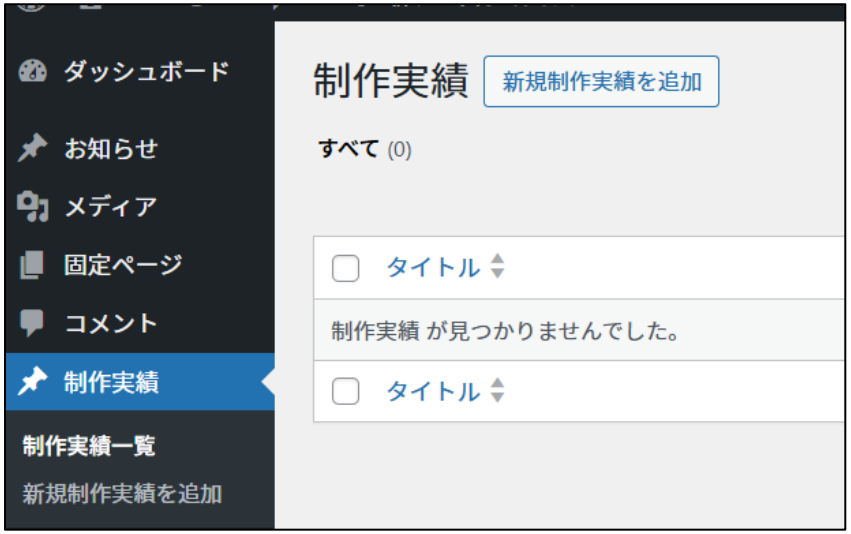



⑥最下部の「投稿タイプを追加」を押します。



## カスタム投稿タイプの追加

⑦サイドメニューに「制作実績」が増えているので、通常の投稿と同じように新規で投稿を作成して公開します。




⑧「/works」が新しく作成した投稿タイプの一覧ページのURLになります。

## カスタム投稿タイプのアーカイブページ

カスタム投稿タイプの記事は出ましたが、ページのデザインがお知らせの一覧ページと同じになっています。これは、テンプレートファイルが「archive.php」しかないためです。
カスタム投稿タイプを追加する際に設定したスラッグに合わせてテンプレートファイルを作成します。

通常は、お知らせとカスタム投稿タイプは別のHTML構造になることが多いですが
今回は練習なのでarchive.phpを複製して、archive-works.phpを作成しましょう。
ページタイトルを制作実績に変更しましょう。

```
<h1 class="page_title">制作実績</h1>
```



Re start

## カスタムタクソノミーについて

「タクソノミー」とは分類のことです。通常の投稿に最初からついている「カテゴリー」や「タグ」もタクソノミーの一種です。

| タクソノミー名 | スラッグ |
| --- | --- |
| カテゴリー | category |
| タグ | tag |

＋

「カスタムタクソノミー」

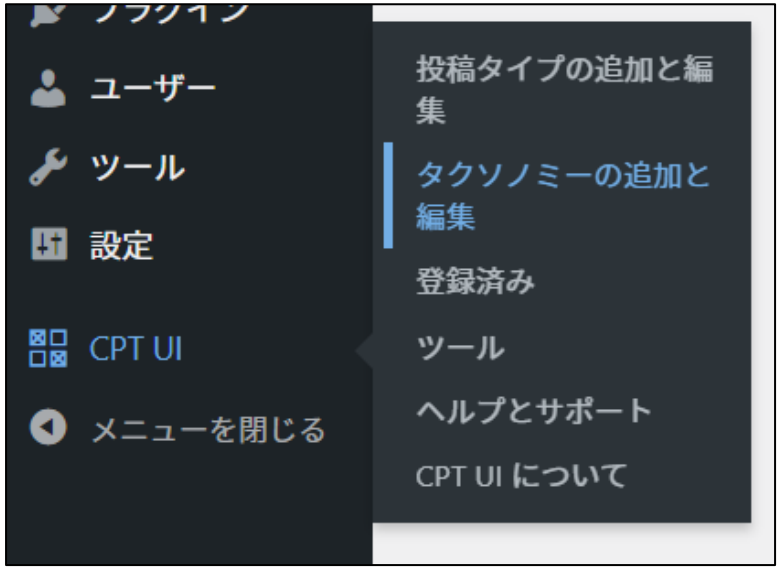
自分で増やしたタクソノミーのことを「カスタムタクソノミー」といいます。
カスタム投稿タイプにも、分類機能をつけたいときに作成します。



Re start

# カスタムタクソノミーの追加

①タクソノミーの追加と編集を押します。




②タクソノミースラッグは英語、ラベルは表示に使用する日本語を設定します。

# カスタムタクソノミーの追加

③設定項目までスクロールし、「階層」「管理画面でカラムを表示」「クイック編集に表示」をTrueにします。

④最下部のタクソノミーを追加を押します。

# カスタムタクソノミーの追加

⑤制作実績のメニュー内に「カテゴリー」が増えるので、いくつかカテゴリーを追加してみましょう。

⑥制作実績の投稿に、カテゴリーを設定しましょう。本文以外の編集には、「クイック編集」が便利です。

## 記事に付与されたタームの取得

archive-works.phpを編集します。記事の取得ループの中の、「記事についたカテゴリー」の箇所を下記のように書き換えましょう。

```
<!-- 記事についたカテゴリー -->
<ul class="news_category">
    <?php
    $terms = get_the_terms( get_the_ID(), 'works-category' );
    foreach ( $terms as $term ) {
        echo '<li">' . esc_html( $term ->name ) . '</li>';
    }
    ?>
</ul>
```

get_the_terms(記事のID, タクソノミースラッグ)を呼び出すと、指定した記事に付与されているタームが配列で取得できます。
※タームは１つ１つの分類のことです。例）「男」や「女」（性別というタクソノミーにつくった分類）

## 全てのタームの取得

**archive-works.phpを編集します。すべてのタームを取得する関数を２種類紹介します。**

| | |
|---|---|
| wp_list_categories() | カテゴリーをリスト形式で表示　※カスタムタクソノミーの場合は引数に指定をいれます。 |
| get_terms() | タクソノミーをオブジェクト配列で取得 |



Re start

## wp_list_categories()

archive-works.phpを編集します。「カテゴリーによる絞り込み」の箇所を下記のように書き換えましょう。

```
<!-- カテゴリーによる絞り込み -->
<ul class="category_list">
  <?php wp_list_categories('taxonomy=works-category'); ?>
</ul>
```

引数で、どのタクソノミーを表示するかを指定します。



Re start

## get_terms()

get_terms()を記述してみましょう。前ページの処理のはコメントアウトしてもかまいません。

```
<ul class="category_list">
    <?php

    $all_terms = get_terms( array(
        'taxonomy' => 'works-category'
    ) );
    foreach ( $all_terms as $term ) {
        echo '<li><a class="category_link" href="' . esc_url( get_term_link( $term )). '">' . esc_html( $term-
>name ) . '</a></li>';
    }
    ?>
</ul>
```



Re start

## タクソノミーアーカイブテンプレートについて

タームの1つをクリックしてタクソノミーアーカイブページに遷移してみましょう。
表示された記事は制作実績の投稿ですが、ページタイトルが「お知らせ」になっていると思います。タクソノミーアーカイブ用のテンプレートがないために、archive.phpが表示に使われている状態だからです。

archive-works.phpと同じ見た目にするために、 archive-works.phpを複製してテンプレートファイルを作ります。

ファイル名は、taxonomy-{taxonomy}.php　とします。
※ {taxonomy}には、タクソノミー名をいれます。

今回作成したのは、タクソノミーは「works-category」なので、
taxonomy- works-category.php　としてください。

## 現在選択中のタームにクラスを付与する

カテゴリーの場合と同じように、現在のタームを取得してID比較を行います。

```
<ul class="category_list">
    <?php
    $current_cat_id = get_queried_object_id();//現在のカテゴリーIDを取得
    $all_terms = get_terms( array(
        'taxonomy' => 'works-category'
    ) );
    foreach ( $all_terms as $term ) {
        $class = ($term->term_id == $current_cat_id) ? 'current-cat' : '';
        echo '<li class="'. $class .'"><a class="category_link" href="' . esc_url( get_term_link( $term )) . '">' .
esc_html( $term->name ) . '</a></li>';
    }
    ?>
</ul>
```

## メインクエリについて

例えば、
・お知らせ詳細ページ（/post/123）
・カテゴリーページ（/category/event）
・カスタム投稿タイプのアーカイブページ（/works）
にアクセスしたとします。このときWordpressは、それぞれのページに表示する内容をデータベースから自動的に取得します。これをメインクエリといいます。

下記の処理はメインクエリで取得した内容を表示する処理ですが、
実行するページによって表示される内容が変わるのは、メインクエリのおかげです。

```
<?php if ( have_posts() ) : ?>
  <?php while ( have_posts() ) : the_post(); ?>

  <?php endwhile;?>
<?php endif;?>
```

## サブクエリ（カスタムクエリ）について

メインクエリと別に、任意の場所で投稿を表示したいという場面もあるはずです。
例えば、
・お知らせ詳細ページで、関連記事を表示する
・TOPページでお知らせの最新に加えて、制作事例も数件表示する
などです。

そのような場合に、自分でクエリを作成（サブクエリ）します。

## サブクエリの作り方

サブクエリの作り方はいくつかあります。
TOPページで制作事例を表示するために、front-page.phpに記述してみましょう。
Works_itemというクラス名がついたaタグが、繰り返したい要素です。
その直前に下記のように記述しましょう。

```
$query = new WP_Query([
    'post_type' => 'works',
    'posts_per_page' => 3
]);
```

post_typeには取得したい投稿タイプを指定します。
posts_per_pageには取得件数を指定します。

これ以外にも、投稿の並び順などのオプションがあります。

## サブクエリの作り方

取得した記事（投稿オブジェクト）は$queryという変数に代入されています。
（変数名は自由に決められます）

```
if ( $query->have_posts() ) :
  while ( $query->have_posts() ) : $query->the_post();
```

```
 <?php endwhile;?>
 <?php endif;?>
```

ループの書き方は、メインクエリとほとんど同じですが、
have_posts() やthe_post()関数の前にオブジェクトが入っている変数名を記述します。

ループ内でのタイトルやリンクの取得はこれまでのやり方と同じです。
制作事例の画像、リンク、タイトル、カテゴリなどを取得・表示しましょう。



Re start

## 条件分岐タグについて

**条件分岐タグは、今表示しているページが特定の条件に当てはまるかどうかを判定する関数です。**
**特定のページだけバナーを出したいなどのカスタムに便利です。**
**よく使う条件分岐タグを紹介します。**

| 条件分岐タグ | 判定内容 | 例 |
|---|---|---|
| is_front_page() | サイトのフロントページか | / |
| is_single() | 投稿ページか | /post/123 |
| is_page() | 固定ページか | /about |
| is_category() | カテゴリーページか | /category/event |
| is_tax() | カスタムタクソノミーのアーカイブか | /works-category/web |
| is_post_type_archive() | カスタム投稿タイプのアーカイブか | /works/ |
| is_404() | 404ページか | 存在しないURL |

## 条件分岐タグの使い方

footer.phpにお問い合わせページへのバナーを記述し、お問い合わせページにはバナーが出ないようにしてみましょう。
footerタグの上に下記のように記述します。

```
<?php if(!is_page('contact')): ?>
<a href="" class="contact_link">
    <span class="contact_link_text">CONTACT</span>
</a>
<?php endif; ?>
```

お問い合わせページ以外に要素が表示されます。



Re start

## 条件分岐タグの使い方

**条件分岐タグはfunctions.phpでも使うことができます。**
**CSSをページごとに分けて読み込むために下記のように書き換えましょう。**

```
function enqueue_style(){
    // wp_enqueue_style( $handle, $src)
    wp_enqueue_style('reset', get_template_directory_uri() . '/css/reset.css');
    wp_enqueue_style('style', get_template_directory_uri() . '/style.css');
    wp_enqueue_style('header', get_template_directory_uri() . '/css/header.css');
    wp_enqueue_style('footer', get_template_directory_uri() . '/css/footer.css’);

    if (is_front_page()) {
        wp_enqueue_style('top', get_template_directory_uri() . '/css/front-page.css');
    }else if(is_page('about')) {
        wp_enqueue_style('page-about', get_template_directory_uri() . '/css/about.css');
    }else if(is_page('contact')) {
        wp_enqueue_style('page-contact', get_template_directory_uri() . '/css/contact.css');
    }else if(is_post_type_archive('works')) {
        wp_enqueue_style('archive-works', get_template_directory_uri() . '/css/archive-works.css');
    }else if(is_archive()) {
        wp_enqueue_style('archive', get_template_directory_uri() . '/css/archive.css');
    }else if(is_single()) {
        wp_enqueue_style('single', get_template_directory_uri() . '/css/single.css');
    }

}
```

## カスタムフィールドについて

投稿や固定ページに、タイトルや本文、カテゴリなどの基本項目以外に「追加の情報」を自由に持たせるための機能です。
ソースコードを修正しなくても、管理画面から情報を修正できるようするなど自由度がかなりアップします。

カスタムフィールドを作る際は、プラグインを使用するのが便利です。

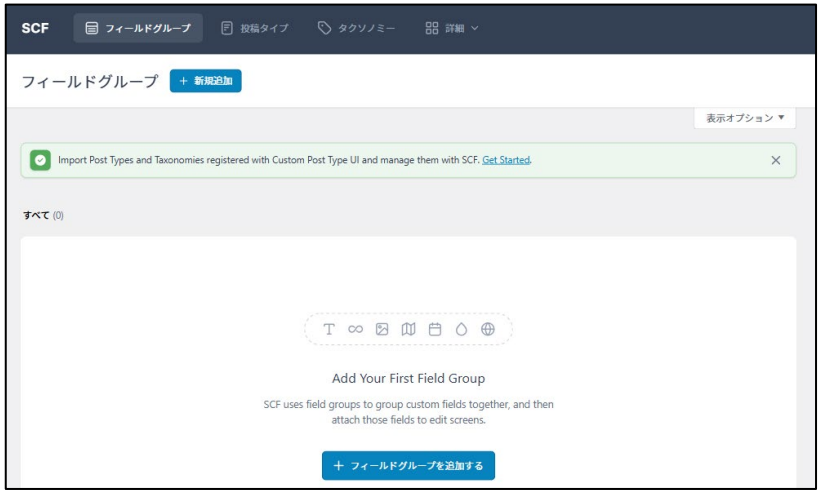
## SCFプラグインインストール

①プラグインの新規追加から検索でSCFと入力し、インストールしてください。

②有効化すると、サイドメニューに「SCF」というメニューが追加されます。

## SCFを使って会社概要を編集可能にする

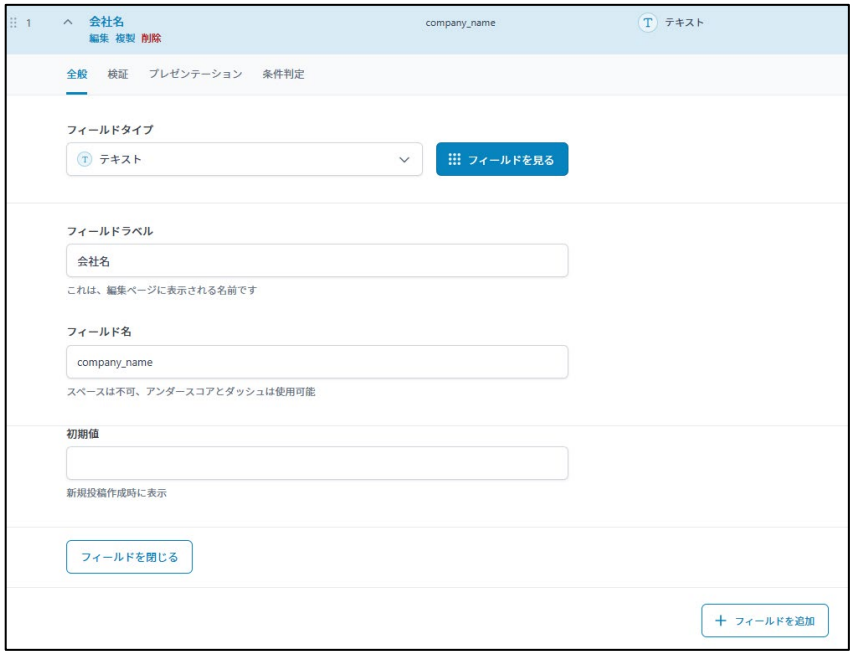
①フィールドグループの新規追加をクリックします。




②フィールドグループタイトルを「会社概要」とします。

## SCFを使って会社概要を編集可能にする

③下記のように入力します。ラベルは管理画面に表示されるため日本語でわかりやすくつけ、フィールド名は英語でつけます。




④右下の「フィールドを追加」をクリックして、必要な項目分作成します。事業内容は複数行入力できるよう、タイプを変更します。

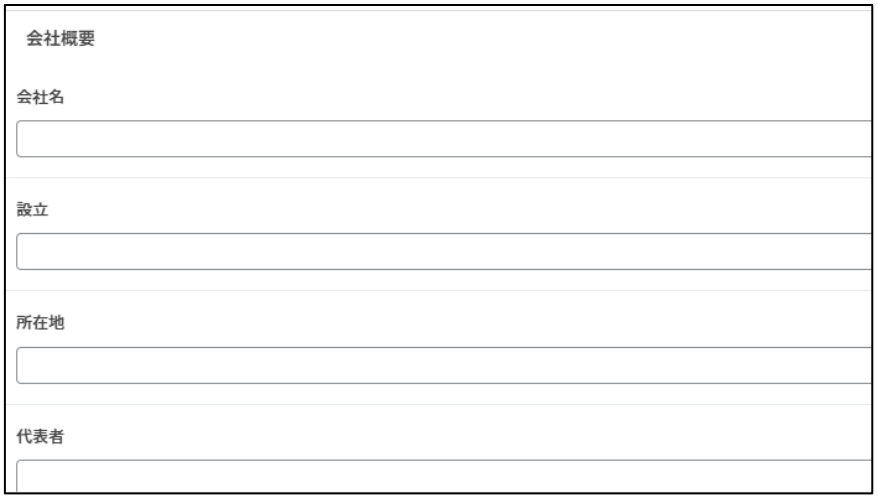
## SCFを使って会社概要を編集可能にする

⑤各フィールドには、「検証」「プレゼンテーション」など設定項目があります。テキストエリアは管理画面での改行が反映されるよう「プレゼンテーション」タブで自動的にbr追加に変更しておくとよいでしょう。

⑥フィールドグループをどこに設置するかを設定します。画像のように設定しましょう。条件は必要に応じて、複数設定可能です。最後に変更を保存します。

## SCFを使って会社概要を編集可能にする

⑦固定ページの会社概要ページ編集画面を開きます。本文入力の下に入力欄が増えていることが確認できます。
内容を入力したら保存します。



## カスタムフィールドを取得・表示する

page-about.php を編集します。フィールドの入力値を表示するには、the_field(‘フィールド名’);を使用します。

```
<tr>
  <th>会社名</th>
  </<td><?php the_field('company_name'); ?></td>
tr>
```

フィールドの入力値を取得するには、get_field(‘フィールド名’);を使用します。

```
<tr>
  <th>会社名</th>
  </<td><?php echo echo get_field('company_name'); ?></td>
tr>
```



Re start

## SCFの様々なフィールドタイプ

テキストやテキストエリア以外にも、画像やラジオボタンなどのタイプが用意されています。
「Secure Custom Fields　使い方」で検索してみましょう。

◆SCF練習課題

①会社概要の表の事業内容の下に、従業員数や電話番号などの項目名も自由に変更でき、数も自由に増やせる状態にする。
ヒント：繰り返しフィールドを使います。

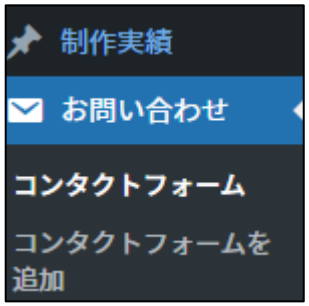
②会社概要ページの表の下に取引先セクションを追加。自由に数を増やし、ロゴ画像、社名、サイトURLを入力可能にする。

EX
制作事例詳細（single-works.php)を作成し、本文のパーツを自由に選んで記事を作れるようにする。
ヒント：柔軟なコンテンツを使います。

## Contactプラグインインストール

①プラグインの新規追加から検索でContact Form 7と入力し、インストールしてください。

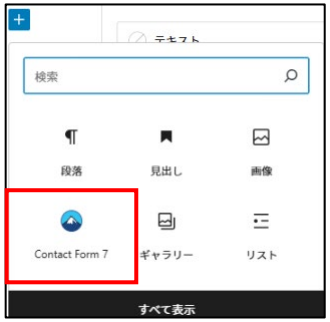
②有効化すると、サイドメニューに「お問い合わせ」というメニューが追加されます。



# Contact formの編集

③page-contact.phpの<form>タグの中身をコンタクトフォーム１の入力欄に貼り付けます。
formの開始・修了タグは削除してください。

④固定ページのお問い合わせページ編集で、本文にcontact formブロックを追加し、プルダウンで先ほど編集したコンタクトフォーム１を選びます。



## Contact formの表示

⑤page-contactのフォームがあった場所に本文取得の処理を書きます。

```
<div class="form_wrap">
  <?php the_content(); ?>
  <!-- <form action="">

    <div class="form_group">
      <label class="form_label" for="contact_name">お
      <input class="form_input" type="text" name="co
    </div>

    <div class="form_group">
      <label class="form_label" for="contact_name_ka
      <input class="form_input" type="text" name="co
    </div>
```

⑥プラグインは自動でpタグを挿入してしまうため、表示が崩れてしまいます。functions.phpに下記のコードを記述します。

```
/*=====================================
 * Contact Form 7のpタグを削除
=====================================*/
function wpcf7_p_remover(){
  return false;
}
add_filter('wpcf7_autop_or_not', 'wpcf7_p_remover');
```



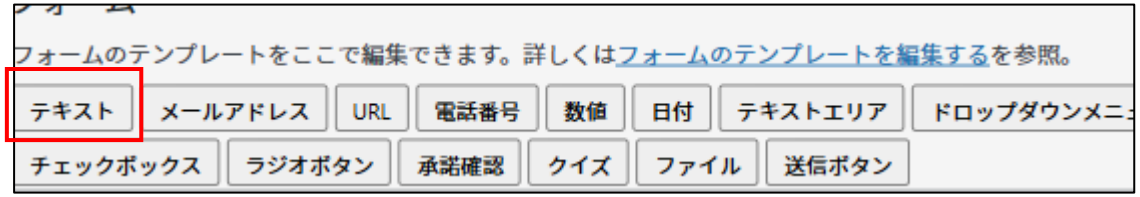
Re start

## Contact form7独自のショートコード

**プラグインの機能を使うために、inputタグを独自のショートコードに書き換えます。**

```
<input class="form_input" type="text" name="contact_name" id="name" placeholder="田中 太郎">
```

**上記のコードを削除し、その行にカーソルがある状態で、「テキスト」ボタンをクリックします。**



## Contact form7独自のショートコード

下の画像のように入力します。全てのタグを同じように作成したら保存します。

必須にする場合にチェックを入れる。

項目名はname属性を記載
※氏名やメールアドレスなど特別な入力欄には追加でチェックをいれます。

クラス属性はinputタグに付与したいクラスを指定。

デフォルト値にはプレースホルダーにしたい文字を入力。

## メールの設定

**メールタブを選択すると、自動送信メールの設定ができます。**

メール

メールのテンプレートをここで編集できます。詳しくはメールのセットアップを参照。
以下の項目にて、これらのメールタグを利用できます:
[contact_name] [contact_name_kana] [e-mail] [category] [message]

**使用されているinputのname属性が一覧で表示されています。このタグを使用してメールの本文を作成します。**

メッセージ本文

ホームページからお問い合わせがありました。

---
名前：[contact_name]
ふりがな：[contact_name_kana]
メールアドレス：[e-mail]
種別：[category]
お問い合わせ内容
[message]

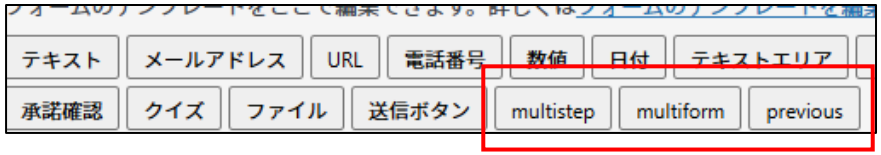


Re start

## Multi-Step Formsプラグインインストール

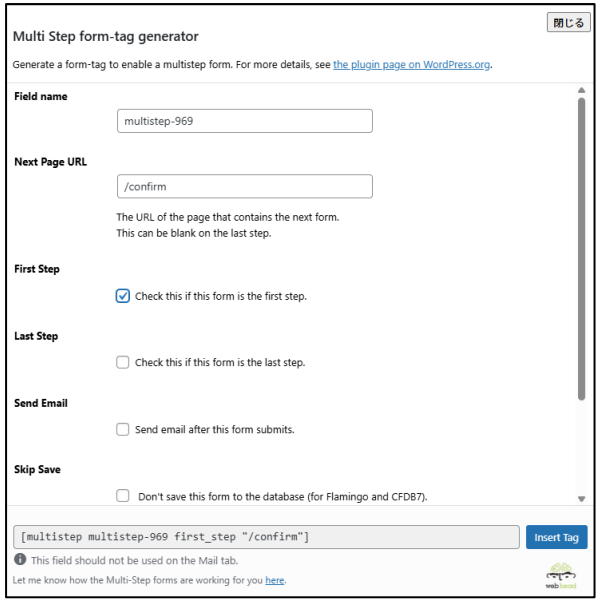
①プラグインの新規追加から検索でmultistepと入力し、インストールしてください。

②有効化したら、お問い合わせメニューから先ほど作成したフォームの編集画面を開きます。新たに３つのタグが追加されています。



## 確認画面用のフォーム作成

③コンタクトフォーム１の編集画面を開きます。submitタグの下の行にカーソルがある状態で、multistepタグボタンをクリックします。画像のように入力します。



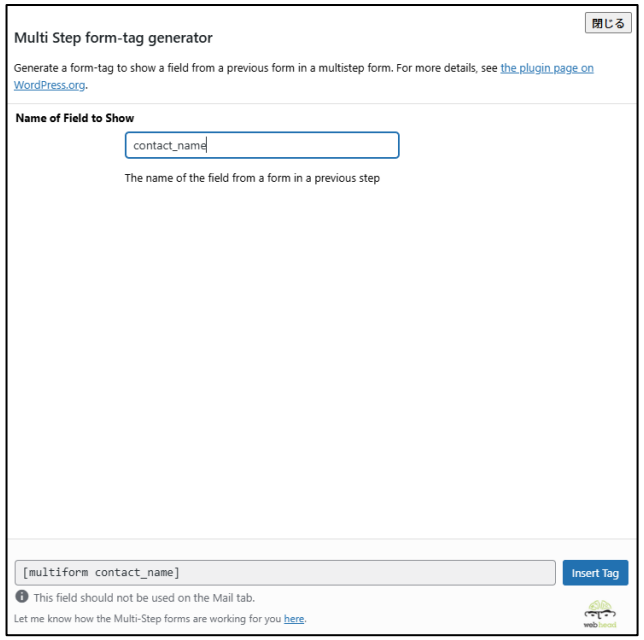

④このようになったら保存します。

```
<div class="btn_wrap">
  [submit class:form_btn]
  [multistep multistep-969 first_step "/confirm"]
</div>
```

# 確認画面用のフォーム作成

⑤コンタクトフォーム１を複製し、タイトルを「確認画面」に変更します。

⑥inputタグを消して、その行でmultiformタグボタンを押します。



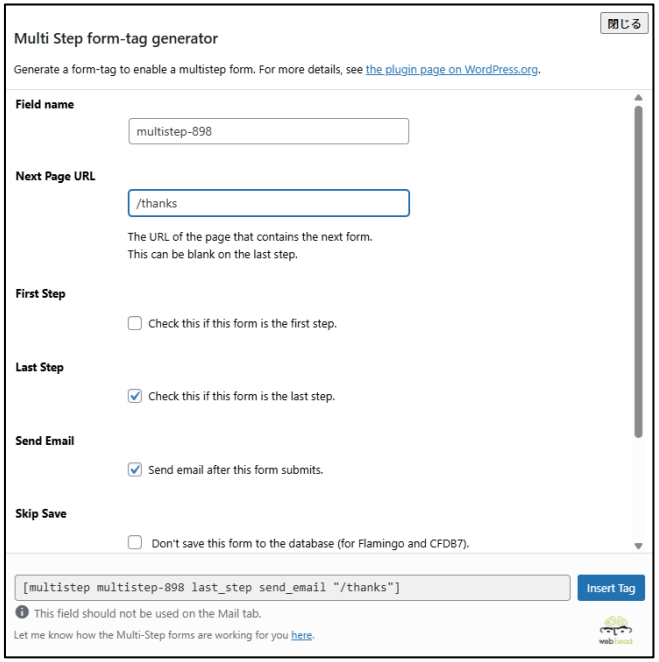
## 確認画面用のフォーム作成

⑦すべての入力欄を画像のように書き換えます。Submitのしたのmultistepタグを削除して、再度multisteptタグボタンをクリックします。

承諾確認 | クイズ | ファイル | 送信ボタン | multistep | multiform | previous

```
<div class="form_group">
  <label class="form_label" for="contact_name">お名前*</label>
  [multiform contact_name]
</div>
```

⑧確認画面のURLの入力とLast StepとSend Emailのチェックを入れます。



## 確認画面用のフォーム作成

⑨submitボタンの上の行で、previousタグボタンをクリックします。画像のようにしたら保存します。

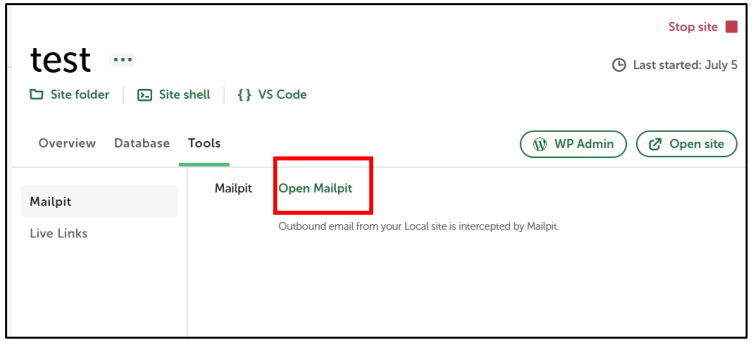
```
</div>

<div class="btn_wrap">
  [previous previous-110]
  [submit class:form_btn]
  [multistep multistep-898 last_step send_email "/thanks"]
</div>
```

⑩固定ページを新規追加し、タイトルを確認画面、スラッグをconfirmにします。Contact form7ブロックを追加して、確認画面を選択します。page-contact.phpを複製して、page-confirm.phpを作りましょう。

## 確認画面用のフォーム作成

⑪固定ページを新規追加し、タイトルをお問い合わせ完了、スラッグをthanksにします。page-thanks.phpも作成しましょう。

⑫お問い合わせページで動作確認をしましょう。
管理画面に届くメールは、LocalのToolsタブから確認できます。



## All In One SEOプラグインインストール

①プラグインの新規追加から検索でSEOと入力し、インストールしてください。

②サイドメニューの設定をクリックします。サイトタイトルとキャッチフレーズに入力します。

# All In One SEOプラグインインストール

③サイドメニューの検索の外観をクリックします。

④サイドメニューの検索の外観をクリックします。区切りとメタディスクリプションを変更します。

Re start